UNIT 5
DevOps Trends

Divya Thakur
28300

Syllabus

- DevOps Market Trends
- DevOps Engineer Skills
- DevOps Delivery Pipeline
- DevOps Ecosystem
- Role of a DevOps Engineer
- Devops Tools:

Git, Docker, Selenium, Maven, Jenkins, Puppet,Ansible, Kubernetes, Nagios

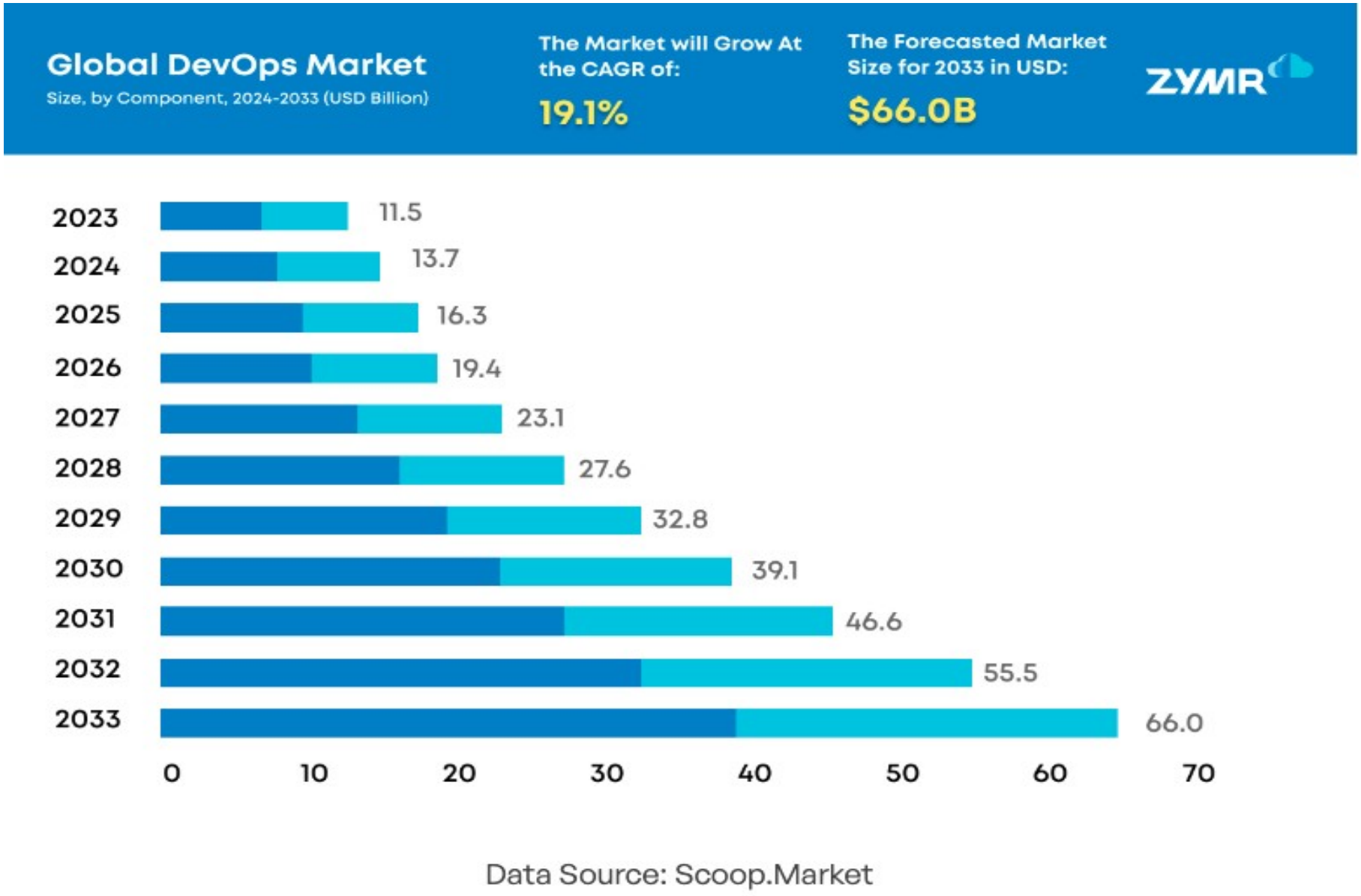
Devops

- DevOps, a combination of Development and Operations, has become a crucial aspect of software development in recent years.
- It focuses on collaboration, communication, and integration between software developers and IT operations teams to ensure a smooth and efficient software delivery process.
- DevOps is expected to grow substantially in the coming years, with an anticipated annual increase of 25% between 2024 and 2032.

- Integrating AI and ML into the software development life cycle is a major growth driver.
- It improves predictive analytics, automated testing, and intelligent monitoring.
- Enhanced productivity remains a core focus, driven by automation of repetitive tasks and streamlined workflows.
- The alignment of DevOps with cloud and microservices architectures is set to continue, offering scalability, resilience, and rapid innovation.
- Quality assurance will be paramount, with rigorous testing and real-time monitoring integrated into DevOps workflows.
- Security will advance with access controls, encryption, and AI-driven enhancements in version control systems.
- To be successful ,DevOps must balance the speed of technology adoption with robust security and quality practices.

- DevOps is ascending in the digital landscape with wide-scale adoption across industries.
- Reports suggest the DevOps market will reach USD 66 billion within a decade.
- 2024 trends see DevOps accommodating technologies like IaC, multi-cloud, AI/ML, and more
- In its security-cognizant version, DevSecOps, DevOps is also making amendments to include security measures within the CI/CD pipelines.

Market analysis



Data Source: Scoop.Market

Devops Market Trends

1. Infrastructure as Code (IaC)
2. Cloud Adoption for DevOps
3. DevSecOps
4. Multi-Cloud Strategies
5. Artificial Intelligence
6. Kubernetes
7. Agile Practices
8. Serverless Architecture
9. Legacy Systems
10. Implementation of ML in DevOps

1. Infrastructure as Code (IaC)

- Infrastructure as code is more than automation and has become essential in DevOps.
- IaC takes infrastructure including servers, networks, and storage devices (on-premises or in the cloud), and it is a means to manage IT infrastructure through configuration files.
- Some of the key benefits of IaC include full speed while setting up a complete infrastructure by running a script, consistency in deploying the same configurations, accountability, traceability, and high efficiency during the entire software development cycle.

2. Cloud Adoption for DevOps

- As per the report, cloud-based deployment held a 65.2% share of its market presence in 2023.
- This indicates a cloud adoption trend that is affecting the DevOp strategies in 2024 and beyond. With on-demand resource scaling, flexibility with tool integration, and cost-effective resource management, the cloud offers great advantages for DevOps.
- With comprehensive cloud strategies, the DevOps team can also ensure adherence to agile processes while offering unhindered support for technologies like AI/ML, IoT, SaaS, and more.
- This means that DevOps teams need to be more proactive in their cloud roadmap for better efficiency.
- Cloud-native technology use in DevOps increased by 30% over the past year.

3. DevSecOps

- The report also shows a growing appeal for DevSecOps as security concerns rise in 2024.
- One of the prime reasons for adopting DevOps was its security benefits.
- However, with attacks like ransomware, DDoS, and data breaches affecting businesses across industries, CISOs readily embrace the DevSecOps model for its “security first” approach.
- While there are DevSecOps that integrates security and compliance testing into the development pipelines.
- It provides swiftness in speed and agility in security mechanisms, prompts responses towards change, and leads to quick detection of vulnerabilities and bugs in code.
- Despite a lot of ongoing myths around DevSecOps, CISOs are focusing on its security benefits for their modern organizations in 2024.
- DevOps teams dedicate 54% of their time to observability, monitoring, and security.

4. Multi-Cloud Strategies

- Among the many cloud considerations in DevOps, the primary has to be multi-cloud.
- There is intense competition among the leading cloud providers—AWS, Azure, and Google Cloud.
- However, the appeal of AWS comes with concerns about vendor lock-in.
- Therefore, efforts from Azure and GCP to offer multi-cloud capabilities are being seen as a positive step by many businesses.
- This means that DevOps 2024 is also welcoming multi-cloud strategies for agility and cost-optimization.
- Most use public clouds for DevOps, with many also adopting hybrid and multi-cloud strategies.

5. Artificial Intelligence

- The DevOps team can use AI and ML to facilitate their workflow.
- Businesses are already making a move to AIOps to optimize the DevOps environment for AI adoption.
- Handling and managing big data is a cumbersome task that requires an AI-driven approach in DevOps.
- AI has now become a valuable tool that predominantly helps in the decision-making process in DevOps.
- AI can assist DevOps through data accessibility has always been a significant concern.
- AI provides data seamlessly to the DevOps team as and when required in no time.
- Adoption of DevOps automation tools like Jenkins and Ansible went up by 25%.

6. Kubernetes

- With Kubernetes, the developers can easily share different software and applications with the IT operations team in real-time.
- The primary reason for bugs and errors is the difference in the IT environment or infrastructure.
- Kubernetes helps in combating that hindrance and promotes collaboration and effectiveness between both teams.
- Kubernetes is helping boost DevOps efficiency by improving development and accommodating features like continuous testing for faster time to market.

- The survey conducted by Atlassian suggests positive sentiment for DevOps by 99% of businesses.
- This can be seen in correlation with businesses' emphasis on agile methodologies.
- In other words, 2024 continues to witness a symbiotic relationship between DevOps and agile practices.
- Deployment frequency increased by 30% over traditional methods, reflecting higher agility.

8. Serverless Architecture

- The Agile DevOps team helps organizations when they are on the verge of migrating from traditional IT structure to Serverless Architecture, especially in the initial phases when IT support is of utmost need.
- After the migration gets done to the serverless platform, the DevOps team is left with minimal maintenance work.
- DevOps is the only successful way to complete the migration process.
- The duo goes hand in hand with similar objectives and complements each other on various fronts.

9. Legacy Systems

- Legacy systems and infrastructures are still being modernized owing to the integration and security challenges that arise during DevOps implementation.
- In 2024, with the acceptance for new technologies and innovations and the advent of DevSecOps, businesses are being more aggressively modernized for DevOps.
- This modernization is also being done to fulfill compliance requirements as companies plan to expand.

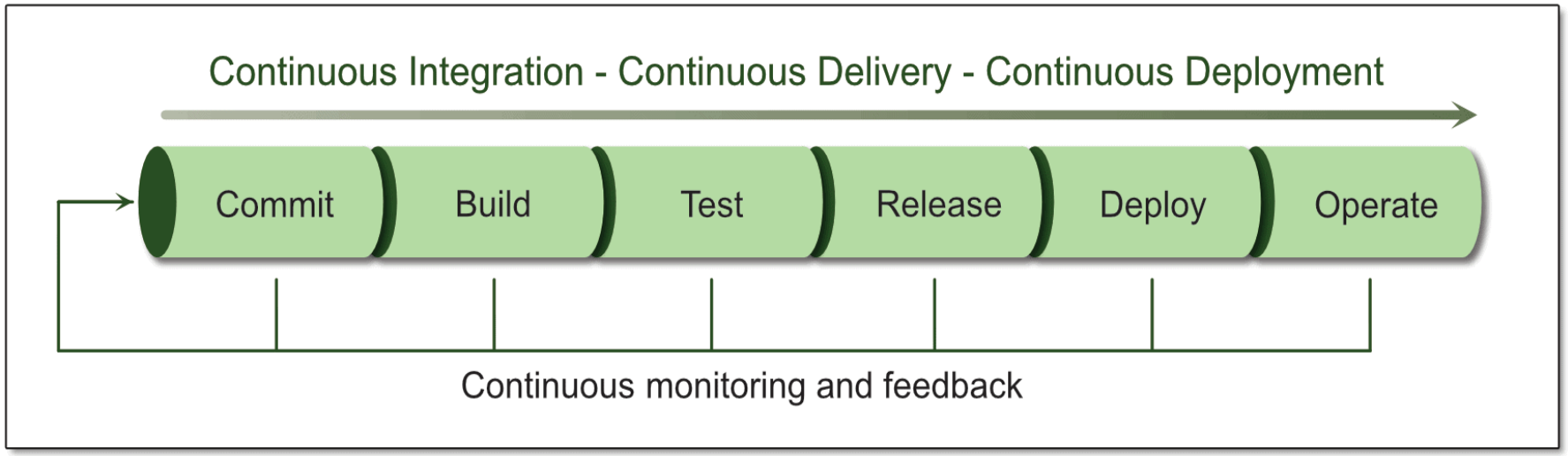
10. Implementation of ML in DevOps

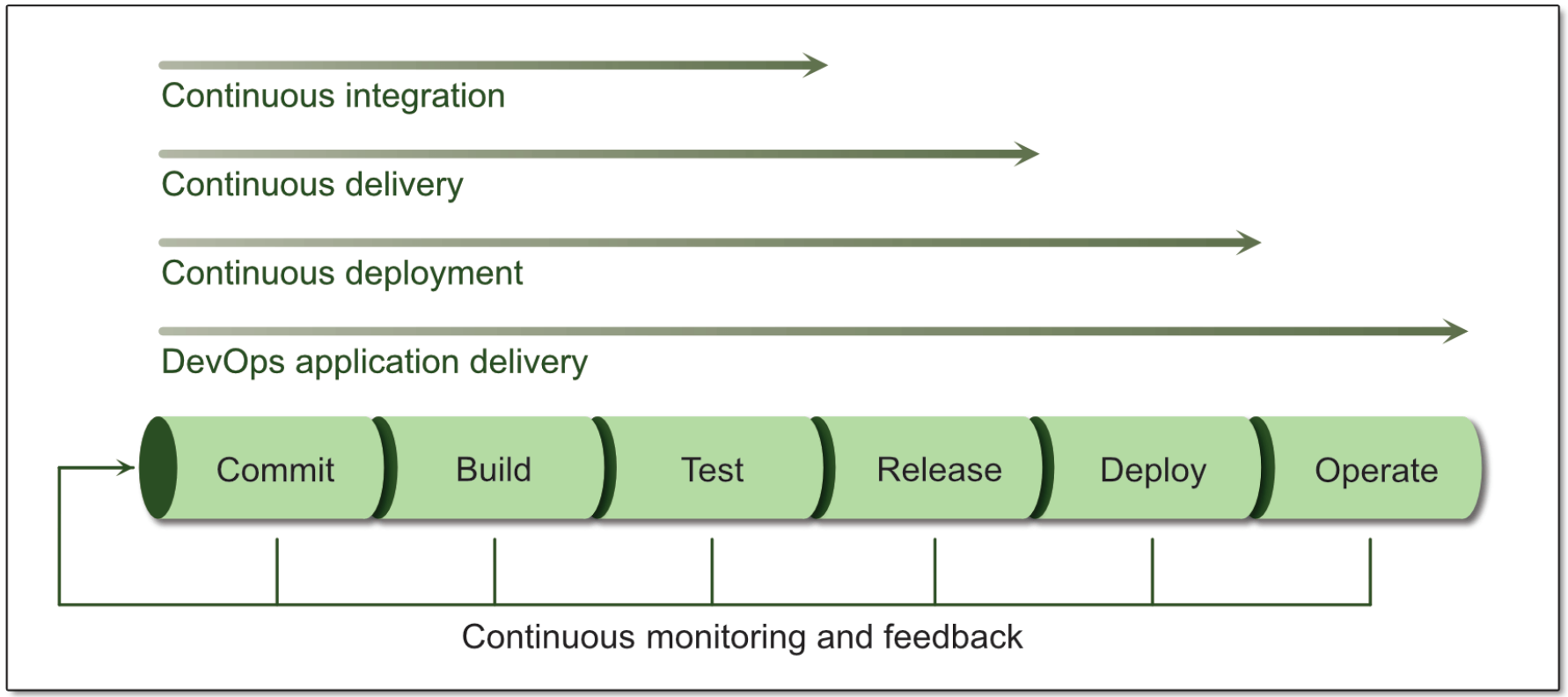
- We already talked about AI, but machine learning too is finding its own space in DevOps.
- The challenges in deploying and managing ML models are being met by MLOps.
- Extending DevOps for ML models, MLOps is ensuring that the CI/CD benefits are accommodative of model training, monitoring, and governance.
- Therefore, DevOps 2024 will see more integration with MLOps frameworks.

Devops Engineer Skills

DevOps Delivery Pipeline

- A DevOps pipeline is a set of automated processes and tools that allows developers and operations professionals to collaborate on building and deploying code to a production environment.
- It is a series of automated steps that all the code changes go through from development to production.
- This pipeline consists of a series of interconnected stages, each performing a specific task.


Continuous Integration - Continuous Delivery - Continuous Deployment
Commit
Build
Test
Release
Deploy
Operate
Continuous monitoring and feedback


Continuous integration
Continuous delivery
Continuous deployment
DevOps application delivery
Commit
Build
Test
Release
Deploy
Operate
Continuous monitoring and feedback

- These stages typically include:

1. Version Control:
   - DevOps follows the GitOps trend.
   - It is a special approach within DevOps that focuses on managing infrastructure and application development by storing them in Git repositories.
   - This makes it easier for everyone to collaborate and reduces errors.
   - It also enables teams to easily roll back changes to previous versions in case of issues, minimizing recovery time.
2. Continuous Integration (CI):
   - Whenever a developer commits a set of updated code, it triggers a set of automated builds and tests.
   - This ensures early detection of bugs and prevents regressions from breaking the codebase.

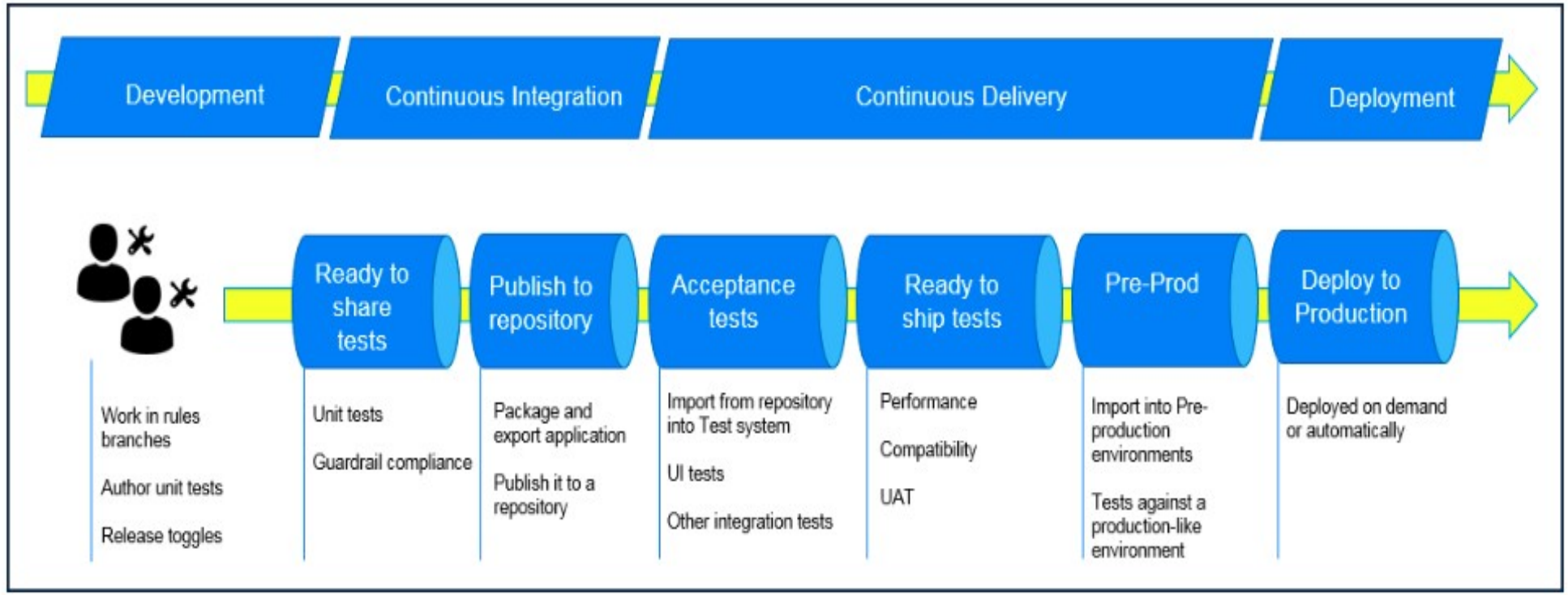
3. Continuous Delivery (CD):
   - Once code passes CI, it's automatically packaged and prepped for deployment.
   - There is minimal manual intervention and it promotes faster release cycles.
4. Continuous Deployment:
   - The pipeline automates deployment without the need for manual approval.
   - This reduces the risk of errors associated with manual deployments, plus no time is wasted promoting faster releases.
5. Continuous Operations:
   - Post-deployment monitoring ensures the application functions as expected.
   - This includes tracking performance metrics, identifying errors, and facilitating quick rollbacks if necessary.

6. Continuous testing :

- It is a critical component of every DevOps pipeline and one of the primary enablers of continuous feedback.
- In a DevOps process, changes move continuously from development to testing to deployment, which leads not only to faster releases, but a higher quality product.
- This means having automated tests throughout your pipeline, including unit tests that run on every build change, smoke tests, functional tests, and end-to-end tests.

7. Continuous monitoring

- It is another important component of continuous feedback.
- A DevOps approach entails using continuous monitoring in the staging, testing, and even development environments.
- It is sometimes useful to monitor pre-production environments for anomalous behavior, but in general this is an approach used to continuously assess the health and performance of applications in production.


Development
Continuous Integration
Continuous Delivery
Deployment
Ready to share tests
Publish to repository
Acceptance tests
Ready to ship tests
Pre-Prod
Deploy to Production
Work in rules branches
Author unit tests
Release toggles
Unit tests
Guardrail compliance
Package and export application
Publish it to a repository
Import from repository into Test system
UI tests
Other integration tests
Performance
Compatibility
UAT
Import into Pre-production environments
Tests against a production-like environment
Deployed on demand or automatically

Benefits of Delivery Pipelines

1. Increased Speed and Efficiency
2. Improved Quality
3. Enhanced Collaboration
4. Better Resource Management

Types of Delivery Pipelines

- There are various delivery pipeline approaches, each catering to specific needs:

1. Continuous Delivery (CD Pipeline)
    - This is where the code is automatically packaged after it has passed all tests and is prepped for deployment. The developer just needs to approve and deploy the already-prepped package.
2. Continuous Deployment (CD Pipeline with Automatic Production Deployments)
    - This is an extension of CD. This process eliminates the need for manual approval and schedules automatic deployments. In this case, the code is pushed to production when it has passed all the previous stages in the pipeline.

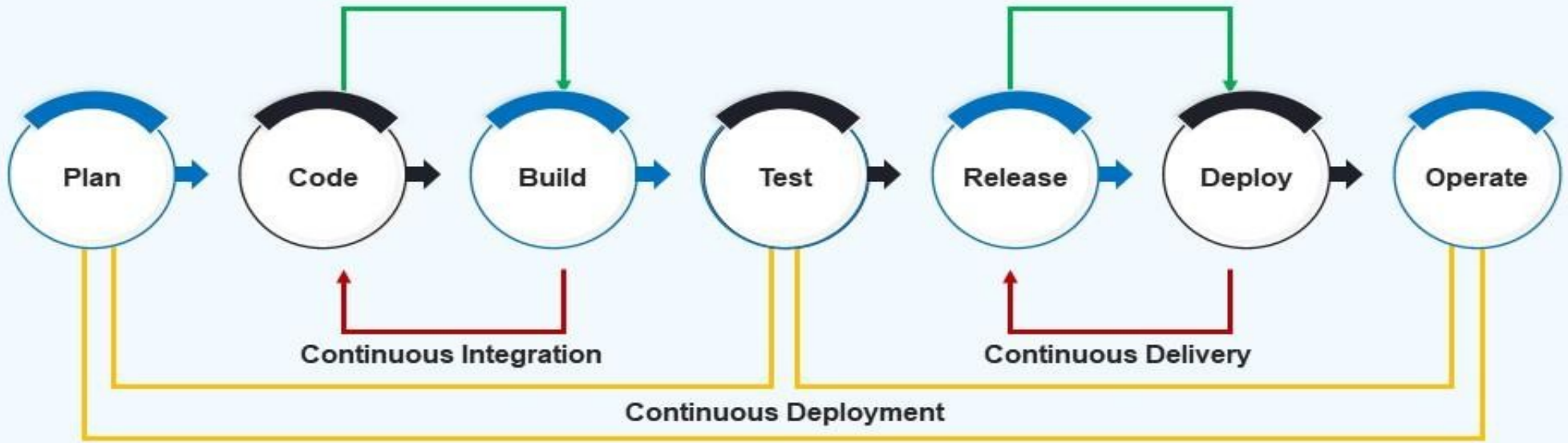
3. Feature Flag Pipelines

- These pipelines allow some control over who gets access to the new features.
- Features can be toggled on or off for specific user groups, facilitating A/B testing.

4. GitOps Pipelines

- This approach uses Git as the single source of truth.
- This means this approach believes the latest update on the Git repository.
- It focuses on managing infrastructure and application development stored in the Git repositories.
- This makes it easier for everyone to collaborate and reduces errors. All the changes are made through Git commits. This enables teams to easily roll back changes to previous versions in case of issues, minimizing recovery time. The Git repository has the updated infrastructure and application configurations.

This slide displays workflows of DevOps pipeline along with the details of its phases. Phases covered are plan, code, build, test, release, deploy and operate.



Planning

- Requirement assessment and finalization
- Task assignment
- Timeline finalization
- Add text here

Code

- Configuration finalization
- Static-code analysis
- Automated and peer review
- Add text here

Build

- Code compilation
- Unit testing
- Create and update dashboards
- Add text here

Test

- Load and stress test
- UI testing
- Penetration testing
- Add text here

Release

- Release notes preparation
- Version tagging
- Feature freeze
- Add text here

Deploy

- Infrastructure updating
- Deployment verification
- Add text here
- Add text here

Operate

- Dashboard monitoring
- Monitoring of error logs
- Alarm triggers
- Add text here

DevOps Ecosystem

Role of a DevOps Engineer

- A DevOps Engineer Expert has an essential role in integrating the project functions and resources across the product life cycle, right from planning, building, testing, and deployment to support.
- DevOps Engineers are conversant with all the technical as well as IT operations aspects for integrated operations.
- They are expected to know about the various automation tools which may be required for process automation and testing.
- A DevOps engineer’s roles and responsibilities are a combination of technical and management roles.
- It is essential to have excellent communication and coordination skills to successfully integrate various functions in a coordinated manner and deliver the responsibilities to the customer’s satisfaction.

Some of the core responsibilities of DevOps Engineer include –

- Understanding customer requirements and project KPIs(key performance indicator).
- Implementing various development, testing, automation tools, and IT infrastructure
- Planning the team structure, activities, and involvement in project management activities.
- Managing stakeholders and external interfaces
- Setting up tools and required infrastructure
- Defining and setting development, test, release, update, and support processes for DevOps operation
- Have the technical skill to review, verify, and validate the software code developed in the project.
- Troubleshooting techniques and fixing the code bugs
- Monitoring the processes during the entire lifecycle for its adherence and updating or creating new processes for improvement and minimizing the wastage
- Encouraging and building automated processes wherever possible

- Identifying and deploying cybersecurity measures by continuously performing vulnerability assessment and risk management
- Incidence management and root cause analysis
- Coordination and communication within the team and with customers
- Selecting and deploying appropriate CI/CD tools
- Strive for continuous improvement and build continuous integration, continuous development, and constant deployment pipeline (CI/CD Pipeline)
- Mentoring and guiding the team members
- Monitoring and measuring customer experience and KPIs
- Managing periodic reporting on the progress to the management and the customer

Essential Skills for a DevOps Engineer

- Pre-requisite skills required for a DevOps Engineer role include:

  - Experience working on Linux based infrastructure
  - Excellent understanding of Ruby, Python, Perl, and Java
  - Configuration and managing databases such as MySQL, Mongo
  - Excellent troubleshooting
  - Working knowledge of various tools, open-source technologies, and cloud services
  - Awareness of critical concepts in DevOps and Agile principles

Devops Tools:

DevOps transcends the limitations of the traditional waterfall model. It involves various development, testing, and deployment technologies to establish automated CI/CD pipelines. Below are some well-known DevOps tools:

- **Git and GitHub:** Source code management (Version Control System).
- **Jenkins:** An automation server equipped with plugins for constructing CI/CD pipelines.
- **Selenium:** Automation testing.
- **Docker:** A platform for software containerization.
- **Kubernetes:** A tool for container orchestration.
- **Puppet:** Used for configuration management and deployment.
- **Chef:** Also employed for configuration management and deployment.
- **Ansible:** Another tool for configuration management and deployment.
- Nagios: Utilized for continuous monitoring.

	Tools		
DevOps Phases	Continuous Planning & Development	• GitLab • GIT • TFS • SVN • Mercurial • Jira • BitBucket • Trello	• Maven • Gradle • Confluence • Subversion • Scrum • Lean • Kanban
	Continuous Integration	• Jenkin • Bamboo • GitLab CI	• TeamCity • Travis and CircleCI • Buddy
	Continuous Testing	• JUnit • Selenium • JMeter • Cucumber • TestSigma	• Microfocus UFT • TestNG • Tricentis Tosca • Jasmine
	Continuous Deployment	• Ansible • Chef • Docker • IBM Urban Code • Kubernetes	• Puppet • Go • Vagrant • Spinnaker • ArgoCD
	Continuous Monitoring	• Nagois • Grafa na • Kibana • Prometheus • Logstash • AppDynamics	• ELK Stack • New Relic • Splunk • Sensu • PagerDuty
	Customer Feedback	• Webalizer • W3Perl • ServiceNow • Slack	• Flowdock • Open Web Analytics • Pendo • Qentelli's TED
	Continuous Operations	• Kubernetes	• Docker Swarm

Tools Across DevOps Phases

SIMFORM

Containers

- Containerization is a virtualization technique that packages an application's components into a single, portable unit called a container. This process allows applications to run independently of the host operating system and consistently across different environments.
- Containerization has become a key part of modern software development, especially for cloud-native applications and microservices architectures. It offers several benefits, including:

    - Resource efficiency
    - Containers only require the resources they need, which prevents resource waste.
    - Portability
    - Containers are lightweight and can be easily moved and run in any environment.
    - Rapid deployment
    - Containers can be deployed quickly and scaled easily.
    - Consistency
    - Containers ensure that applications run consistently, regardless of the underlying infrastructure.
    - Containerization platforms like Docker and Kubernetes help with creating, managing, and orchestrating containers.

- Containerization is a software deployment process that bundles an application’s code with all the files and libraries it needs to run on any infrastructure.
- Traditionally, to run any application on your computer, you had to install the version that matched your machine’s operating system.
- For example, you needed to install the Windows version of a software package on a Windows machine.
- However, with containerization, you can create a single software package, or container, that runs on all types of devices and operating systems.

What are containerization use cases?

- The following are some use cases of containerization.

1. Cloud migration

- Cloud migration, or the lift-and-shift approach, is a software strategy that involves encapsulating legacy applications in containers and deploying them in a cloud computing environment. Organizations can modernize their applications without rewriting the entire software code.

2. Adoption of microservice architecture

- Organizations seeking to build cloud applications with microservices require containerization technology.
- The microservice architecture is a software development approach that uses multiple, interdependent software components to deliver a functional application.
- Each microservice has a unique and specific function.
- A modern cloud application consists of multiple microservices. For example, a video streaming application might have microservices for data processing, user tracking, billing, and personalization.
- Containerization provides the software tool to pack microservices as deployable programs on different platforms.

3. IoT devices

- Internet of Things (IoT) devices contain limited computing resources, making manual software updating a complex process. Containerization allows developers to deploy and update applications across IoT devices easily.

- Containerization involves building self-sufficient software packages that perform consistently, regardless of the machines they run on.
- Software developers create and deploy container images—that is, files that contain the necessary information to run a containerized application.
- Developers use containerization tools to build container images based on the Open Container Initiative (OCI) image specification.
- OCI is an open-source group that provides a standardized format for creating container images.
- Container images are read-only and cannot be altered by the computer system.

Container images are the top layer in a containerized system that consists of the following layers.

- ## Infrastructure

  Infrastructure is the hardware layer of the container model. It refers to the physical computer or bare-metal server that runs the containerized application.

- ## Operating system

  The second layer of the containerization architecture is the operating system. Linux is a popular operating system for containerization with on-premise computers. In cloud computing, developers use cloud services such as AWS EC2 to run containerized applications.

- ## Container engine

  The container engine, or container runtime, is a software program that creates containers based on the container images. It acts as an intermediary agent between the containers and the operating system, providing and managing resources that the application needs. For example, container engines can manage multiple containers on the same operating system by keeping them independent of the underlying infrastructure and each other.

- ## Application and dependencies

  The topmost layer of the containerization architecture is the application code and the other files it needs to run, such as library dependencies and related configuration files. This layer might also contain a light guest operating system that gets installed over the host operating system.

What is container orchestration?

- Container orchestration is a software technology that allows the automatic management of containers. This is necessary for modern cloud application development because an application might contain thousands of microservices in their respective containers. The large number of containerized microservices makes it impossible for software developers to manage them manually.

- Benefits of container orchestration
- Developers use container orchestration tools to automatically start, stop, and manage containers. Container orchestrators allow developers to scale cloud applications precisely and avoid human errors. For example, you can verify that containers are deployed with adequate resources from the host platform.

The following are some examples of popular technologies that developers use for containerization.

- Docker
- Docker, or Docker Engine, is a popular open-source container runtime that allows software developers to build, deploy, and test containerized applications on various platforms. Docker containers are self-contained packages of applications and related files that are created with the Docker framework.

- Linux
- Linux is an open-source operating system with built-in container technology. Linux containers are self-contained environments that allow multiple Linux-based applications to run on a single host machine. Software developers use Linux containers to deploy applications that write or read large amounts of data. Linux containers do not copy the entire operating system to their virtualized environment. Instead, the containers consist of necessary functionalities allocated in the Linux namespace.

- Kubernetes
- Kubernetes is a popular open-source container orchestrator that software developers use to deploy, scale, and manage a vast number of microservices. It has a declarative model that makes automating containers easier. The declarative model ensures that Kubernetes takes the appropriate action to fulfil the requirements based on the configuration files.

Containerization compared to virtual machines

- Containerization is a similar but improved concept of a VM. Instead of copying the hardware layer, containerization removes the operating system layer from the self-contained environment. This allows the application to run independently from the host operating system. Containerization prevents resource waste because applications are provided with the exact resources they need.

Git

- Git remains indispensable in software development and DevOps due to its pivotal role in version control, collaborative coding, and efficient project management.
- Git empowers developers to collaborate on codebases, effortlessly creating and merging branches for new features and bug fixes.
- Its distributed nature ensures developers can work seamlessly offline, an increasingly valuable feature in today's remote and distributed work environments.
- Additionally, Git facilitates the tracking of code modifications, making it easier to identify when and why specific changes were made, a critical aspect of maintaining code quality and security.
- Software development is essential in driving innovation and advancing progress, and Git maintains its prominent position as the bedrock of efficient, cooperative, and secure coding methodologies.

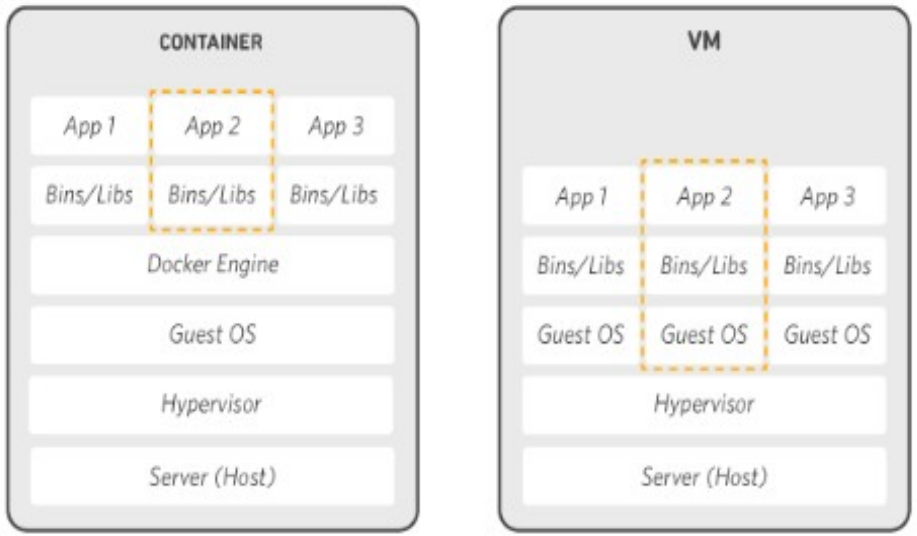
Docker

- Docker is crucial in modern software development and DevOps practices.
- It can simplify and streamline the management of applications across various environments.
- Docker containers encapsulate an app and its dependencies, ensuring consistent and reproducible deployments from development to production.
- This technology enhances portability and scalability, accelerates development cycles, and reduces the "it works on my machine" problem.
- In a rapidly evolving software landscape, Docker's containerization approach remains crucial for achieving efficient, isolated, and highly flexible application deployment, making it an essential component of DevOps and continuous delivery pipelines.

How Docker works

Docker works by providing a standard way to run your code. Docker is an operating system for containers. Similar to how a virtual machine virtualizes (removes the need to directly manage) server hardware, containers virtualize the operating system of a server. Docker is installed on each server and provides simple commands you can use to build, start, or stop containers.

AWS services such as AWS Fargate, Amazon ECS, Amazon EKS, and AWS Batch make it easy to run and manage Docker containers at scale.



Why use Docker

- Using Docker lets you ship code faster, standardize application operations, seamlessly move code, and save money by improving resource utilization.
- With Docker, you get a single object that can reliably run anywhere.
- Docker's simple and straightforward syntax gives you full control.
- Wide adoption means there's a robust ecosystem of tools and off-the-shelf applications that are ready to use with Docker.

When to use Docker

- You can use Docker containers as a core building block creating modern applications and platforms.
- Docker makes it easy to build and run distributed microservices architecures, deploy your code with standardized continuous integration and delivery pipelines, build highly-scalable data processing systems, and create fully-managed platforms for your developers.
- The recent collaboration between AWS and Docker makes it easier for you to deploy Docker Compose artifacts to Amazon ECS and AWS Fargate.

Microservices

Build and scale distributed application architectures by taking advantage of standardized code deployments using Docker containers.

Continuous Integration & Delivery

Accelerate application delivery by standardizing environments and removing conflicts between language stacks and versions.

Data Processing

Provide big data processing as a service. Package data and analytics packages into portable containers that can be executed by non-technical users.

Containers as a Service

Build and ship distributed applications with content and infrastructure that is IT-managed and secured.

Run Docker on AWS

- AWS provides support for both Docker open-source and commercial solutions.
- There are a number of ways to run containers on AWS, including Amazon Elastic Container Service (ECS) is a highly scalable, high performance container management service. Customers can easily deploy their containerized applications from their local Docker environment straight to Amazon ECS.
- AWS Fargate is a technology for Amazon ECS that lets you run containers in production without deploying or managing infrastructure. Amazon Elastic Container Service for Kubernetes (EKS) makes it easy for you to run Kubernetes on AWS.
- AWS Fargate is technology for Amazon ECS that lets you run containers without provisioning or managing servers.
- Amazon Elastic Container Registry (ECR) is a highly available and secure private container repository that makes it easy to store and manage your Docker container images, encrypting and compressing images at rest so they are fast to pull and secure.
- AWS Batch lets you run highly-scalable batch processing workloads using Docker containers.

Service

Amazon ECS

Amazon ECS is a highly scalable, high-performance container orchestration service to run Docker containers on the AWS cloud.

→

Service

AWS Fargate

AWS Fargate is a technology for Amazon ECS that lets you run Docker containers without deploying or managing infrastructure.

→

Service

Amazon EKS

Amazon EKS makes it easy to run Kubernetes on AWS without needing to install and operate Kubernetes masters.

→

Service

Amazon ECR

Amazon ECR is a highly available and secure private container repository that makes it easy to store and manage Docker container images.

→

Service

AWS Batch

AWS Batch enables developers, scientists, and engineers to easily and efficiently run batch computing jobs using containers on AWS.

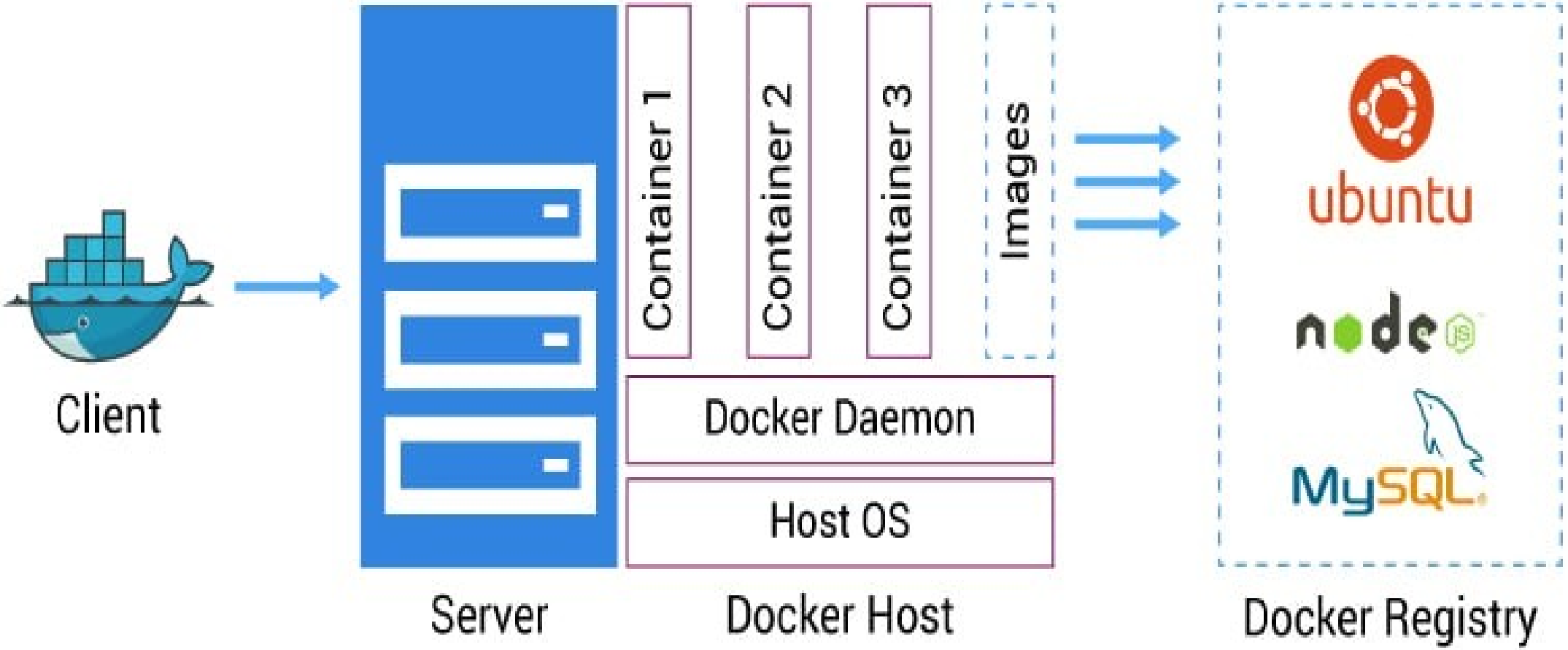
→

Service

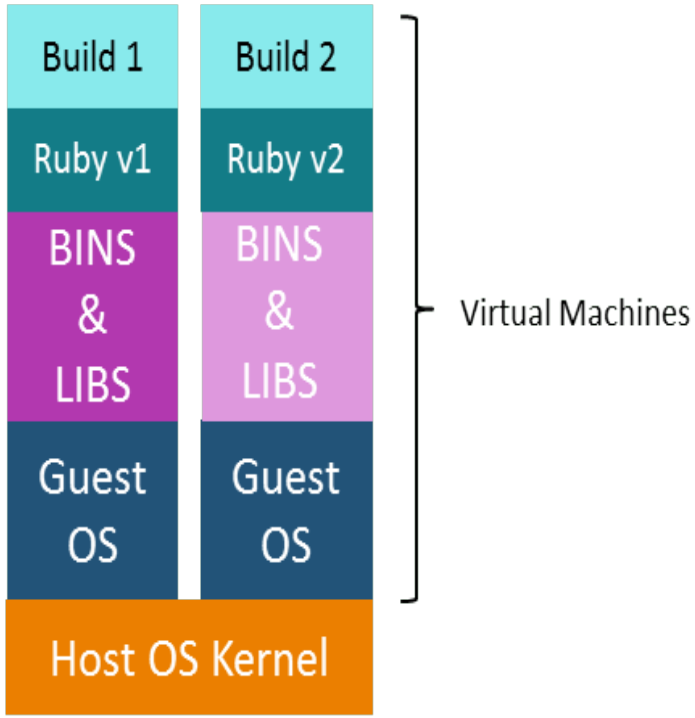
AWS Copilot

AWS Copilot is a command line interface that enables customers to launch and easily manage containerized applications on AWS.

→

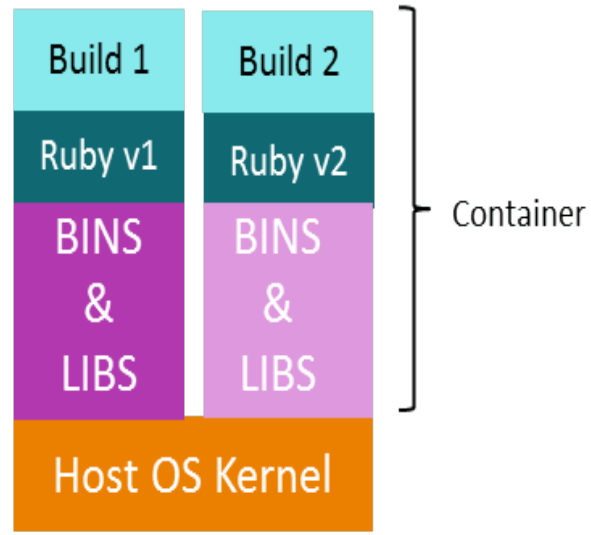

Client
Container 1
Container 2
Container 3
Images
Docker Daemon
Host OS
ubuntu
node
MySQL
Server
Docker Host
Docker Registry

In case of Virtual Machines



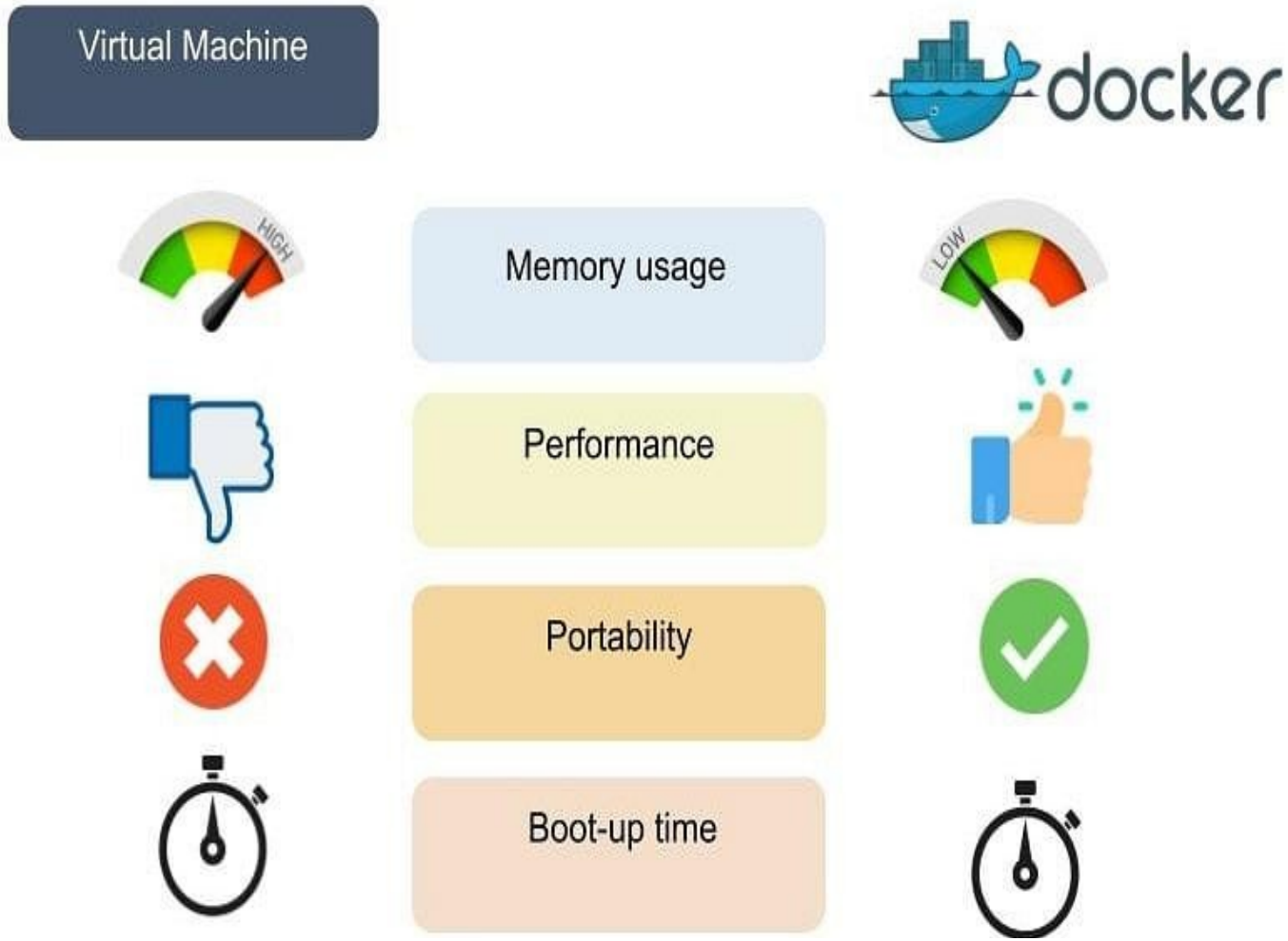
New Builds → Multiple OS → Separate Libraries → Heavy → **More Time**

In case of Docker



New Builds → Same OS → Separate Libraries → Lightweight → **Less Time**

Major differences are:



Link for Docker

- https://aws.amazon.com/getting-started/hands-on/deploy-docker-containers/

Selenium

- It remains a vital tool in software testing and automation due to its enduring relevance in ensuring the quality of web applications.
- As technology evolves, web applications become increasingly complex, requiring thorough testing across various browsers and platforms.
- With its robust automation capabilities and extensive browser compatibility, Selenium allows developers and QA teams to automate repetitive testing tasks efficiently, conduct cross-browser testing, and ensure that web applications function flawlessly across diverse environments.
- Its open-source nature, active community support, and integration with other DevOps tools make Selenium a go-to choice for organizations striving for continuous delivery and the rapid deployment of high-quality software, a cornerstone of modern software development practices

- Selenium is a popular open-source software testing framework used for automating web applications.
- It is widely used for functional testing, regression testing, and performance testing.
- Selenium supports multiple programming languages, including Java, C#, Python, and Ruby, making it accessible to a wide range of developers.
- It was originally developed by Jason Huggins in 2004 as an internal tool at Thought Works.
- Selenium can be easily deployed on platforms such as Windows, Linux, Solaris and Macintosh.
- Moreover, it supports OS (Operating System) for mobile applications like iOS, windows mobile and android.

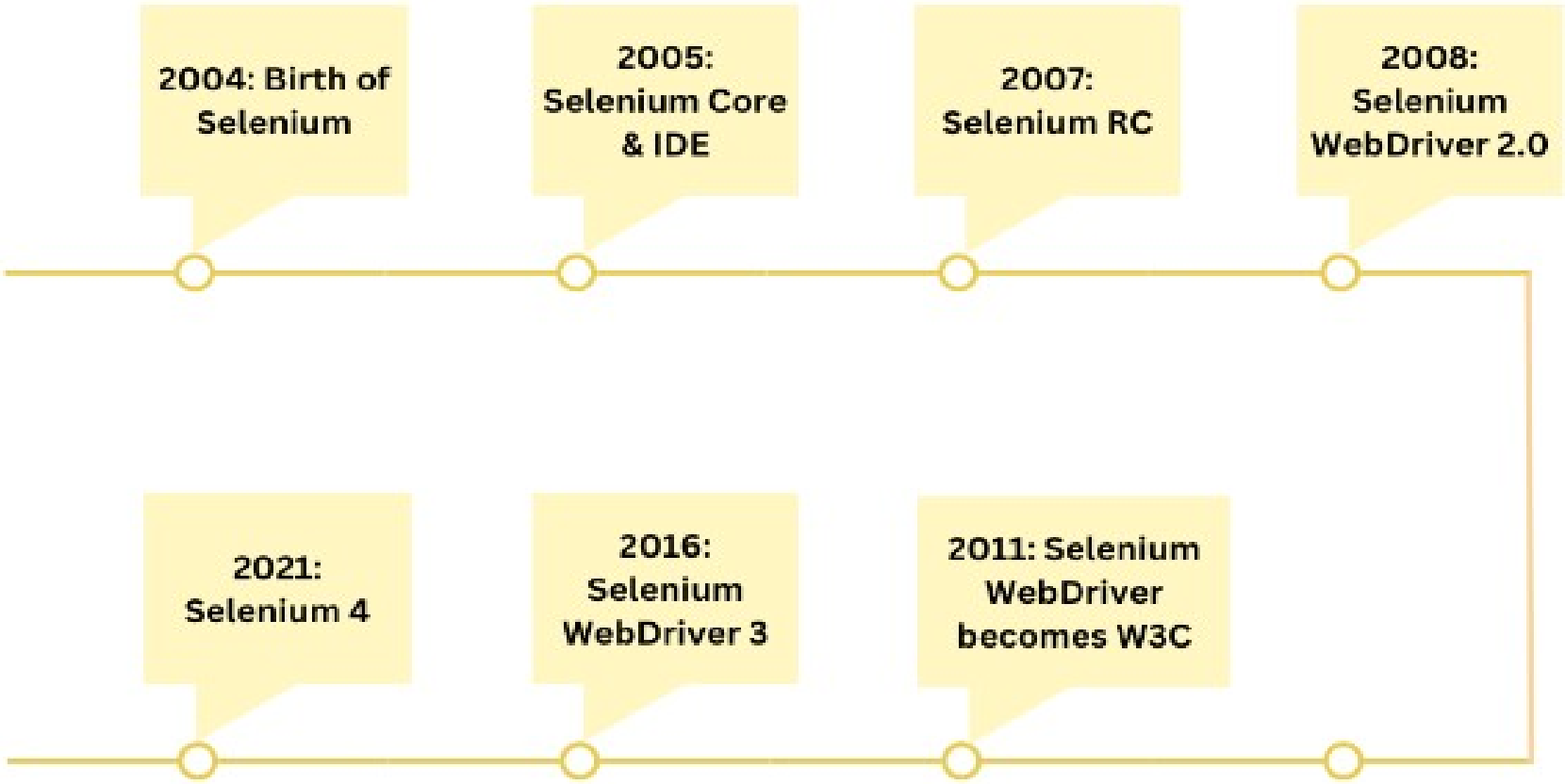
Importance of Testing in Selenium

- Manual testing can be time-consuming and prone to human errors.
- Selenium Automation allows tests to be executed quickly and accurately, reducing the human errors and ensuring consistent test results.
- Importance of selenium :

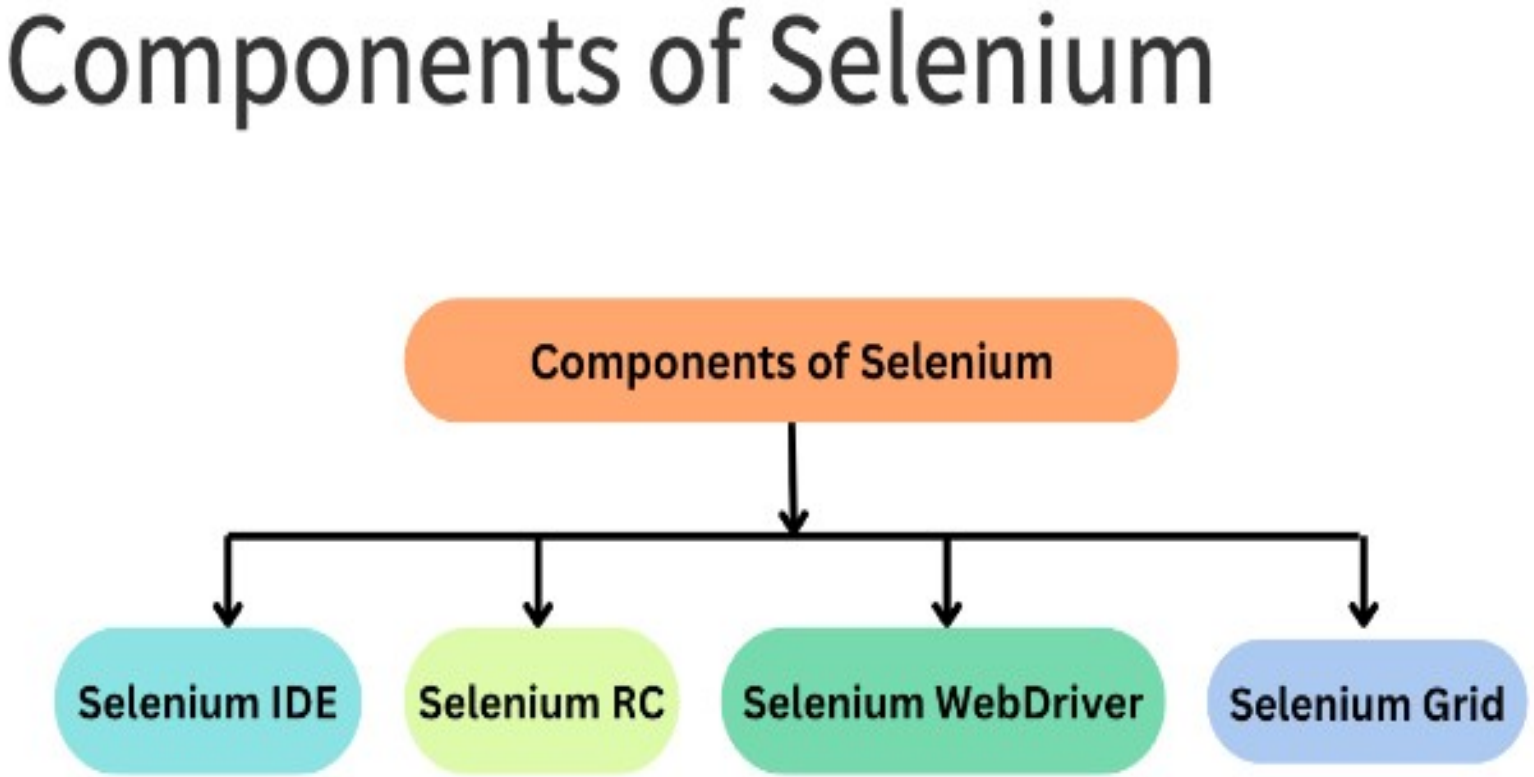
1. Language Support
2. Browser Support
3. Scalability
4. Reusable Test Scripts
5. Parallel Testing
6. Documentation and Reporting
7. User Experience Testing
8. Continuous Integration and Continuous Deployment (CI/CD)

History

The history of Selenium spans several years and involves the development and evolution of a set of tools aimed at automating web testing. Here's a chronological overview of the key milestones in the history of Selenium:



Selenium Components



Selenium IDE

- A GUI-based application with record, edit, and debugging features.
- Selenium IDE is a Chrome and Firefox plugin.
- The primary use of a Selenium IDE is to record user interactions such as clicks, selections etc in the browser and plays them back as automated tests .
- It then generates the test script (of the automated tests) in programming languages like C#, Java, Python, and Ruby and Selenese (Selenium's own scripting language).
- Selenium IDE helps in:
- Creating automated test scripts and validating them at speed
- Identifying and highlighting errors during the replay of interactions
- Cross Browser Testing

Selenium RC

- Selenium RC was built to automate the testing of web applications by simulating user interactions across different browsers and platforms.
- It provided a way to browser automation remotely and execute test scripts written in various programming languages.
- Also known as Selenium 1.0, this was one of the first tools in the Selenium suite.
- It supported multiple programming languages and browsers.

Limitations OF RC

- **Browser Limitations**: Selenium RC had to work with browsers using a JavaScript-based “proxy” mechanism, which introduced potential instability and limitations, especially when working with modern web applications.
- **Speed and Performance**: The use of a JavaScript proxy added overhead and affected the speed and performance of test execution.
- **Maintenance and Compatibility**: Selenium RC required separate “drivers” for each browser, making maintenance and compatibility challenging as browsers continued to update and evolve.
- **Synchronization Issues**: Selenium RC often faced synchronization problems, where test scripts had to wait for the browser to respond before proceeding to the next step.
- **Complex Setup**: Setting up Selenium RC involved multiple components, which could be complex and difficult to configure correctly.

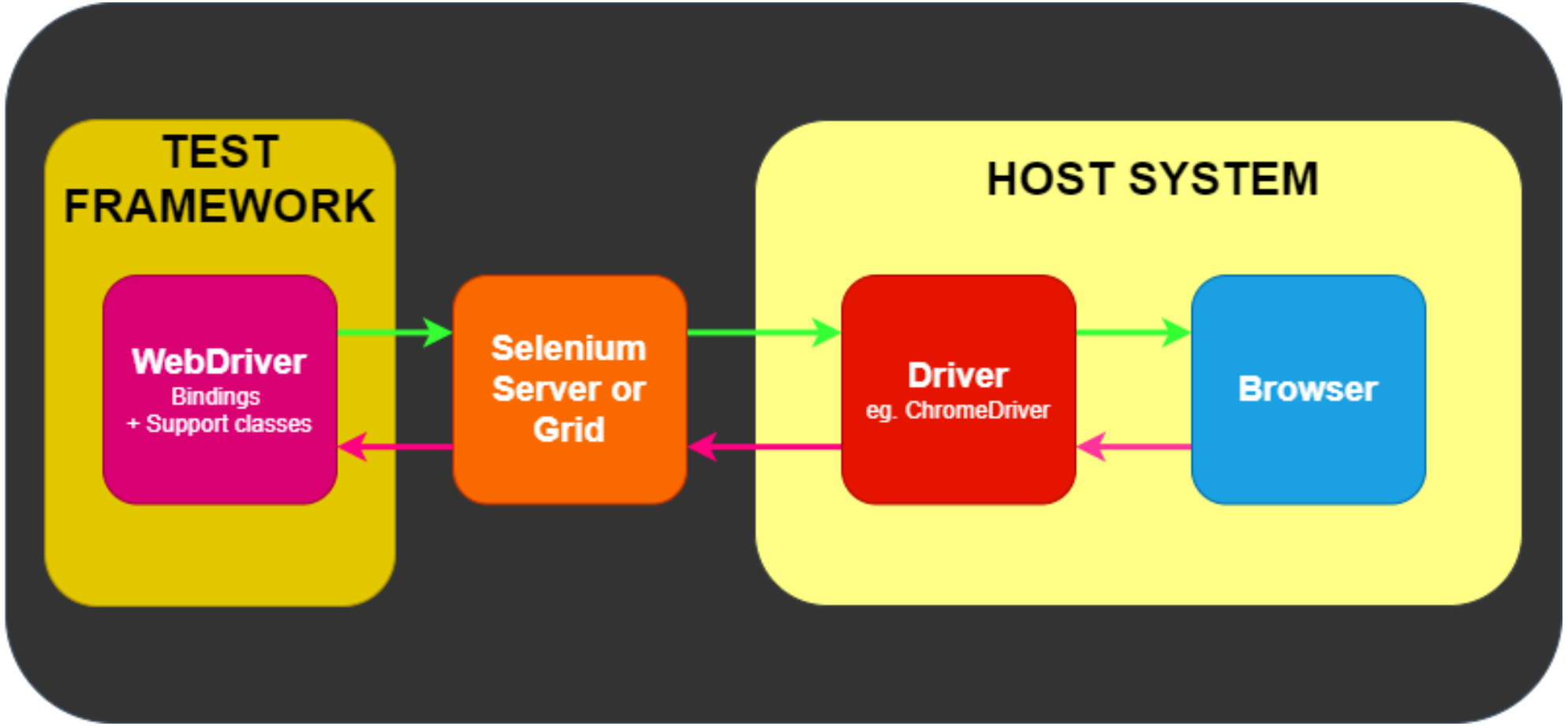
Selenium WebDriver

- Selenium WebDriver is a powerful and enhanced version of Selenium RC which was developed to overcome the limitations of Selenium RC.
- WebDriver communicates with browsers directly with the help of browser-specific native methods, thereby completely eliminating the need of Selenium RC.
- WebDriver works closely with Selenium IDE and Selenium Grid resulting in reliable test execution at speed and scale.
- Also known as Selenium 2.0/3.0, this tool was developed to address some of the restrictions of Selenium RC. It doesn't require a manual method like the Selenium Server.

Selenium Grid

- Selenium Grid is a smart proxy server that allows QAs to run tests in parallel on multiple machines.
- This is done by routing commands to remote web browser instances, where one server acts as the hub.
- This hub routes test commands that are in JSON format to multiple registered Grid nodes.

Architecture



What is Selenium Grid on cloud?

- A Selenium Cloud is basically a Selenium Grid configured on cloud servers. The Cloud Selenium Grid connects to a range of browsers and real devices with different operating systems which are configured and made available 24×7 on-demand. This makes it possible for QA teams to automate tests by executing several test scripts simultaneously on multiple device-browser combinations on the cloud using parallel testing.

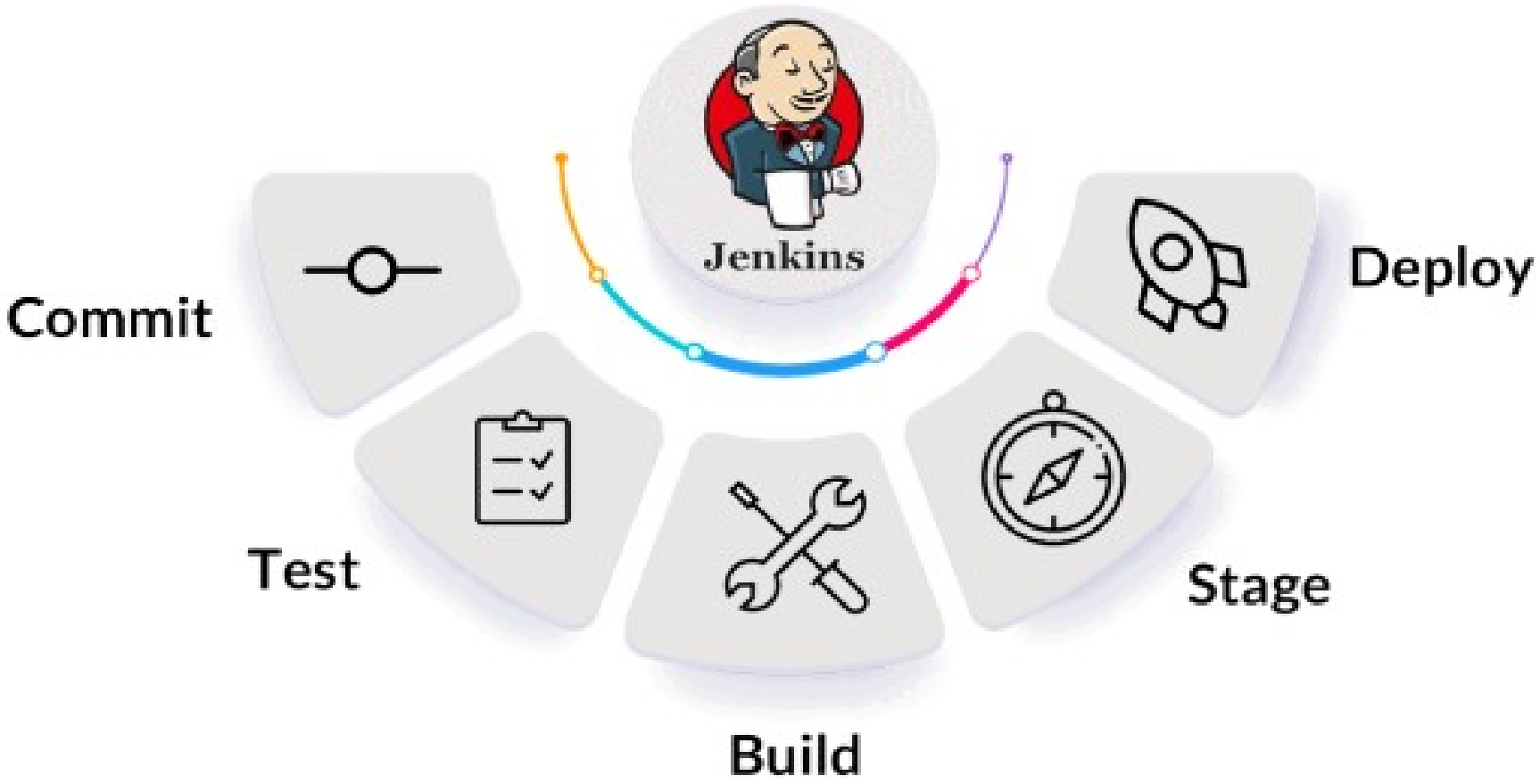
- A Selenium cloud eliminates the overhead of maintaining and updating the physical infrastructure at regular intervals. That means organizations do not have to purchase, maintain, update devices, browsers and operating systems on-site. While maintaining physical devices require a lot of time and effort, buying subscription of a cloud based grid like BrowserStack can be a more feasible solution.

Limitations of Selenium

- Selenium does not support automation testing for desktop applications.
- Selenium requires high skill sets in order to automate tests more effectively.
- Since Selenium is open source software, you have to rely on community forums to get your technical issues resolved.
- We can't perform automation tests on web services like SOAP or REST using Selenium.
- We should know at least one of the supported programming languages to create tests scripts in Selenium WebDriver.
- It does not have built-in Object Repository like UTF/QTP to maintain objects/elements in centralized location. However, we can overcome this limitation using Page Object Model.
- Selenium does not have any inbuilt reportingcapability; you have to rely on plug-ins like JUnit and TestNG for test reports.
- It is not possible to perform testing on images. We need to integrate Selenium with Sikuli for image based testing.
- Creating test environment in Selenium takes more time as compared to vendor tools like UFT, RFT, Silk test, etc.
- No one is responsible for new features usage; they may or may not work properly.
- Selenium does not provide any test tool integration for Test Management.

Jenkins

- Jenkins is used to build and test your product continuously, so developers can continuously integrate changes into the build.
- Jenkins is the most popular open source CI/CD tool on the market today and is used in support of DevOps, alongside other cloud native tools.
- Automation, including CI/CD and test automation, is one of the key practices that allow DevOps teams to deliver “faster, better, cheaper” technology solutions.
- So, Jenkins has become an indispensable tool in helping them achieve those goals.
- Jenkins has been a key enabling technology that is increasingly helping DevOps practices gain widespread adoption in many organizations around the world.


Jenkins
Commit
Test
Build
Stage
Deploy

- Jenkins is a tool that is used for automation.
- It is mainly an open-source server that allows all the developers to build, test and deploy software.
- It is written in Java and runs on java only.
- By using Jenkins we can make a continuous integration of projects(jobs) or end-to-endpoint automation
- Jenkins facilitates the automation of several stages of the software development lifecycle, including application development, testing, and deployment.
- Continuous delivery (CD) and integration (CI) pipelines can be created and managed with Jenkins.
- The development, testing, and deployment of software applications are automated using CI/CD pipelines.
- You will be able to release software more regularly and with fewer problems if you do this.

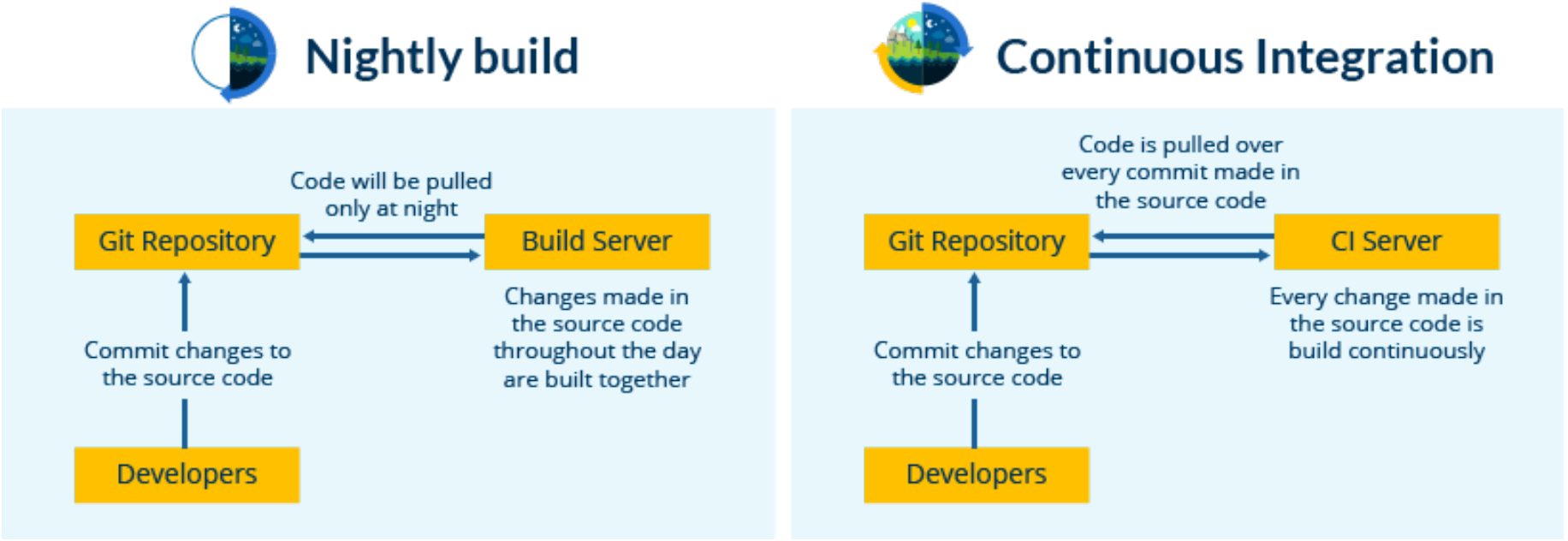
- jenkins is flexible.
- You can add the n no.of plugins you want to add to the jenkins.
- You can automate the proceses of CI/CD pipelines of all the projects.
- There are over a thousand plugins that you can use to extend Jenkins' capabilities and make it more user-specific.
- All of these plugins and extensions are developed in Java.
- This means that Jenkins can also be installed on any operating system that runs on Java.

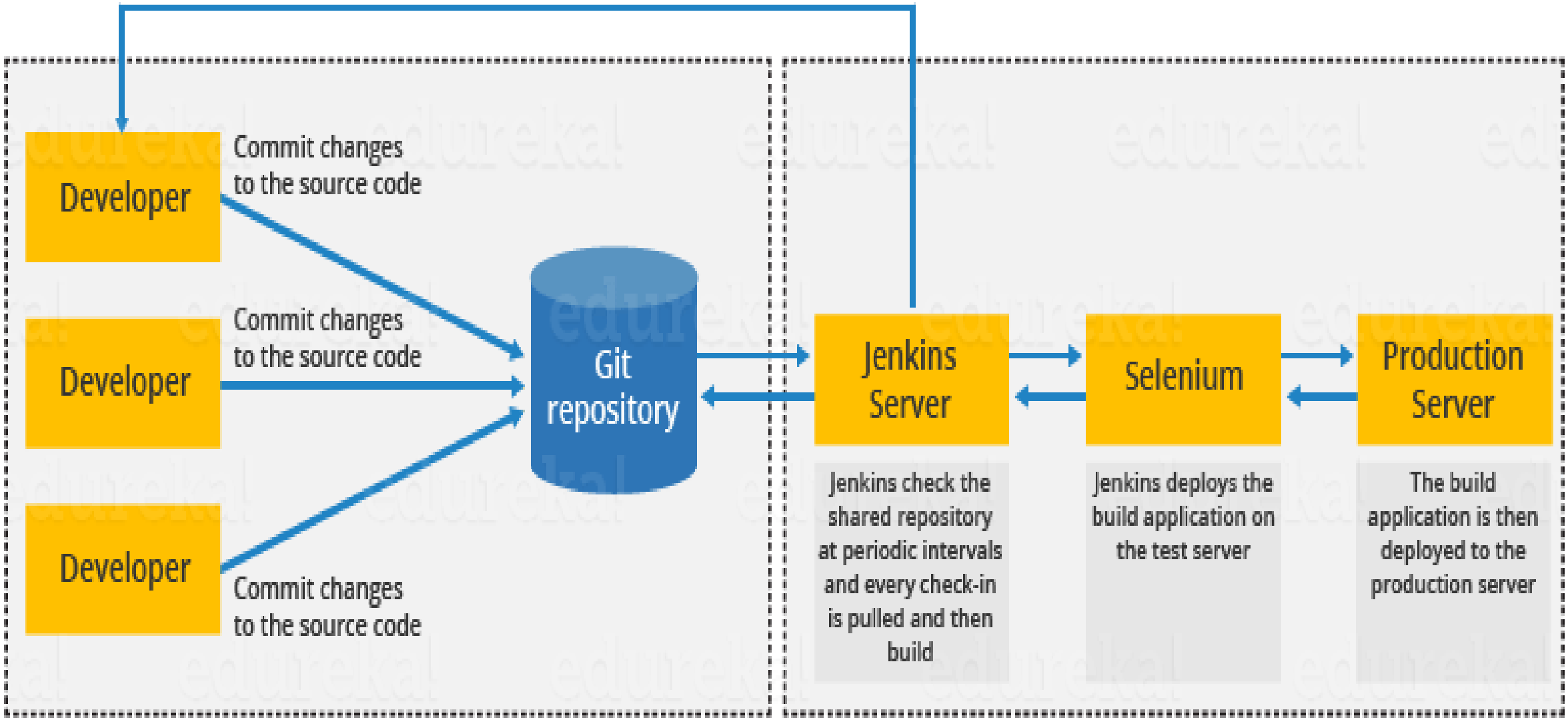
- Its importance lies in its role as a powerful automation server that enables continuous integration and continuous delivery (CI/CD) pipelines.
- With the growing complexity of modern applications, the need for efficient CI/CD processes has become even more paramount.
- Jenkins provides flexibility, extensibility, and a vast library of plugins that cater to a wide range of technologies and tools, making it adaptable to diverse development environments.
- As organizations prioritize speed, reliability, and collaboration in their software development practices, Jenkins stands as a cornerstone tool, enabling teams to achieve seamless automation and efficient delivery of software solutions.

History

- Kohsuke Kawaguchi first developed Hudson in 2004 while working at Sun Microsystems.
- When Oracle acquired Sun Microsystems in 2010, there was a dispute between Oracle and the Hudson community with respect to the infrastructure used.
- There was a call for votes to change the project name from Hudson to Jenkins, which was overwhelmingly approved by the Hudson community on January 29, 2011, thereby creating the first “Jenkins” project.

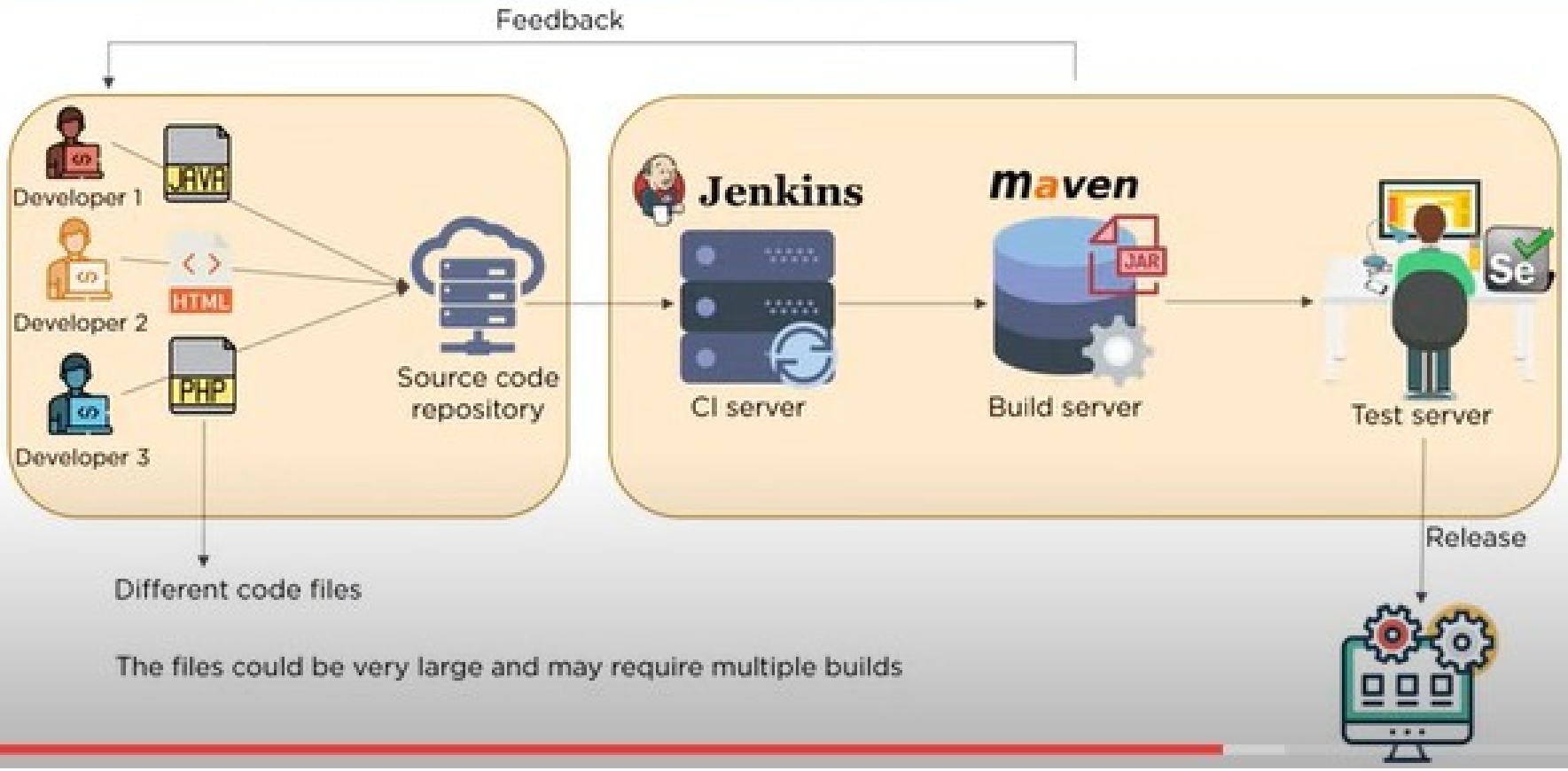
- Organizations may use Jenkins to automate and speed up the software development process. Jenkins incorporates a variety of development life-cycle operations, such as build, document, test, package, stage, deploy, static analysis, and more.




Developer
Commit changes to the source code
Developer
Commit changes to the source code
Developer
Commit changes to the source code
Git repository
Jenkins Server
Jenkins check the shared repository at periodic intervals and every check-in is pulled and then build
Selenium
Jenkins deploys the build application on the test server
Production Server
The build application is then deployed to the production server

- First, a developer commits the code to the source code repository. Meanwhile, the Jenkins server checks the repository at regular intervals for changes.
- Soon after a commit occurs, the Jenkins server detects the changes that have occurred in the source code repository. Jenkins will pull those changes and will start preparing a new build.
- If the build fails, then the concerned team will be notified.
- If built is successful, then Jenkins deploys the built in the test server.
- After testing, Jenkins generates a feedback and then notifies the developers about the build and test results.
- It will continue to check the source code repository for changes made in the source code and the whole process keeps on repeating.

Jenkins Architecture



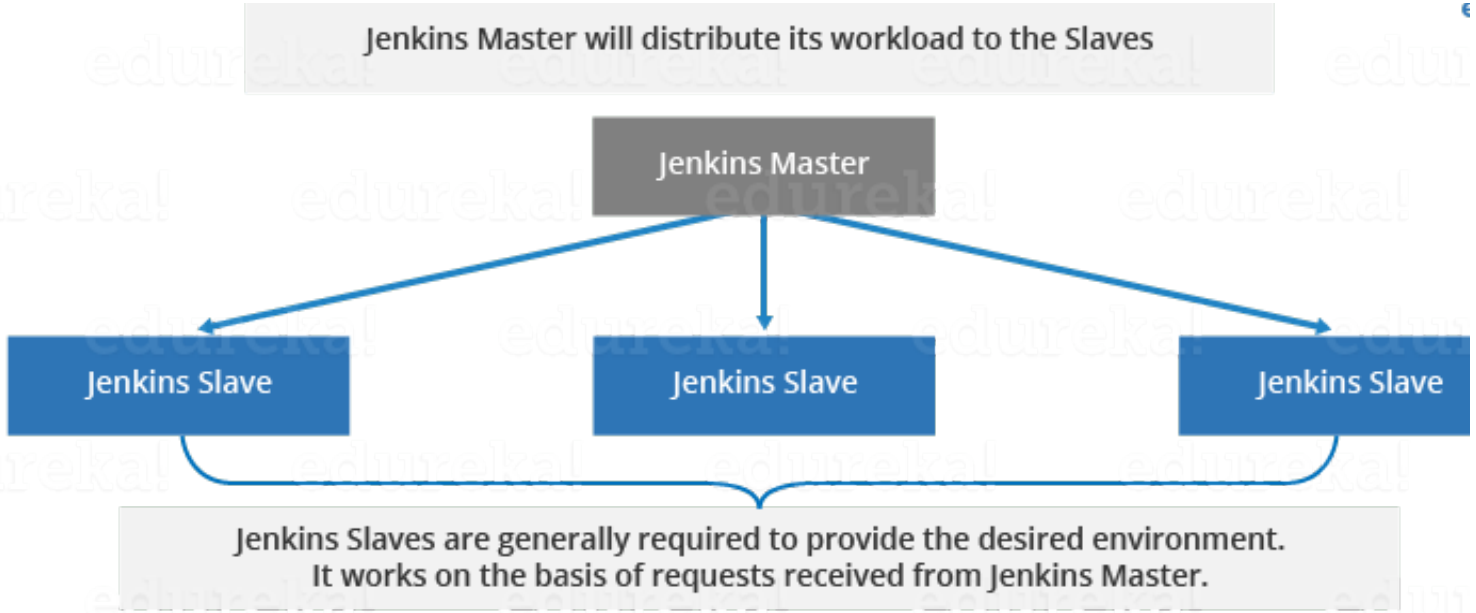
- Here's how Jenkins elements are put together and interact:

- Developers commit changes to the source code, found in the repository.
- The Jenkins CI server checks the repository at regular intervals and pulls any newly available code.
- The Build Server builds the code into an executable file. In case the build fails, feedback is sent to the developers.
- Jenkins deploys the build application to the test server. If the test fails, the developers are alerted.
- If the code is error-free, the tested application is deployed on the production server.
- The files can contain different code and be very large, requiring multiple builds. However, a single Jenkins server cannot handle multiple files and builds simultaneously; for that, a distributed Jenkins architecture is necessary.

Jenkins Master

Your main Jenkins server is the Master. The Master's job is to handle:

- Scheduling build jobs.
- Dispatching builds to the slaves for the actual execution.
- Monitor the slaves (possibly taking them online and offline as required).
- Recording and presenting the build results.
- A Master instance of Jenkins can also execute build jobs directly.

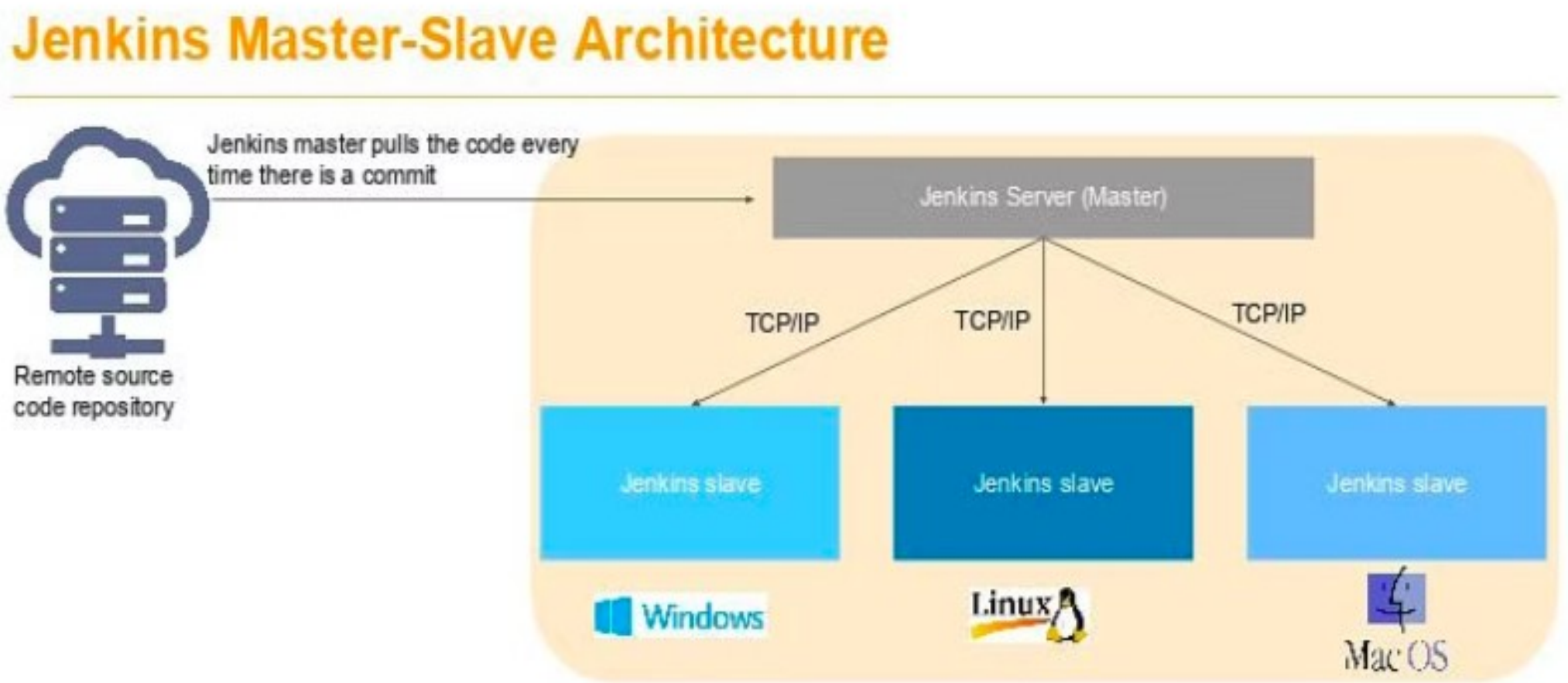


Jenkins Slave

- A Slave is a Java executable that runs on a remote machine.

- The following are the characteristics of Jenkins Slaves:
- It hears requests from the Jenkins Master instance.
- Slaves can run on a variety of operating systems.
- The job of a Slave is to do as they are told to, which involves executing build jobs dispatched by the Master.
- You can configure a project to always run on a particular Slave machine or a particular type of Slave machine, or simply let Jenkins pick the next available Slave.

Architecture



Before and After Jenkins

Before Jenkins	After Jenkins
The entire source code was built and then tested. Locating and fixing bugs in the event of build and test failure was difficult and time-consuming, which in turn slows the software delivery process.	Every commit made in the source code is built and tested. So, instead of checking the entire source code developers only need to focus on a particular commit. This leads to frequent new software releases.
Developers have to wait for test results	Developers know the test result of every commit made in the source code on the run.
The whole process is manual	You only need to commit changes to the source code and Jenkins will automate the rest of the process for you.

Advantages of Jenkins include:

- It is an open-source tool with great community support.
- It is easy to install.
- It has 1000+ plugins to ease your work. If a plugin does not exist, you can code it and share it with the community.
- It is free of cost.
- It is built with Java and hence, it is portable to all the major platforms.

Jenkins Features

- **Adoption:** Jenkins is widespread, with more than 147,000 active installations and over 1 million users around the world.
- **Plugins:** Jenkins is interconnected with well over 1,000 plugins that allow it to integrate with most of the development, testing and deployment tools.

Features of Jenkins

Jenkins Applications

1. Continuous Integration (CI):

Jenkins is primarily used to automate the process of continuous integration, where it builds and tests code every time a change is committed to a version control system. This helps developers detect issues early in the development cycle, improving code quality and reducing the time needed to validate and release new software updates.

Automated Builds: Jenkins can compile and build code from various environments and languages.

Automated Testing: It can run a suite of tests (unit, integration, system) on new code to ensure it doesn't break anything.

2. Continuous Delivery (CD)
   - Beyond continuous integration, Jenkins can automate steps in software delivery, making it easier to deploy and release new versions. It allows for the automation of the deployment process, making sure that you can release reliably at any time.

   - Automated Deployment: Jenkins can automate the deployment of applications to various environments, including testing, staging, and production.
   - Rollbacks: It can also automate the rollback of a deployment if the deployment fails, ensuring quick recovery from errors.

3. Infrastructure as Code (IaC)

- Jenkins is used to implement Infrastructure as Code practices, which involve managing and provisioning infrastructure through code instead of through manual processes.

- Configuration Management: Jenkins can integrate with tools like Ansible, Chef, and Puppet to automate the configuration of servers.
- Server Provisioning: It can also use scripts or templates to create or update servers.

4. Monitoring and Reporting

- Jenkins can be configured to monitor its own performance and to generate reports on various aspects of the development process.

- Build Monitoring: Jenkins can keep track of build success rates and notify developers of failures.
- Performance Trends: It can generate reports that track performance metrics over time, helping teams understand trends.

5. DevOps and Multibranch Pipeline

- Jenkins supports DevOps practices by enabling teams to implement multibranch pipelines, where each branch of the version control system can have its own tailored CI/CD pipeline.

- Pipeline as Code: Jenkins Pipelines allow defining build, test, and deploy stages that are stored in a Jenkinsfile and versioned along with the code.
- Parallel Execution: Jenkins can execute jobs in parallel, reducing the time required for builds and tests.

6. Containerization Support

- Jenkins has robust support for Docker and Kubernetes, allowing teams to use containers for builds, tests, and deployments.

- Docker Integration: Jenkins can build Docker images and push them to Docker registries.
- Kubernetes Integration: It can manage Kubernetes pods, enabling dynamic provisioning of agents for builds and tests.

7. Third-Party Integration

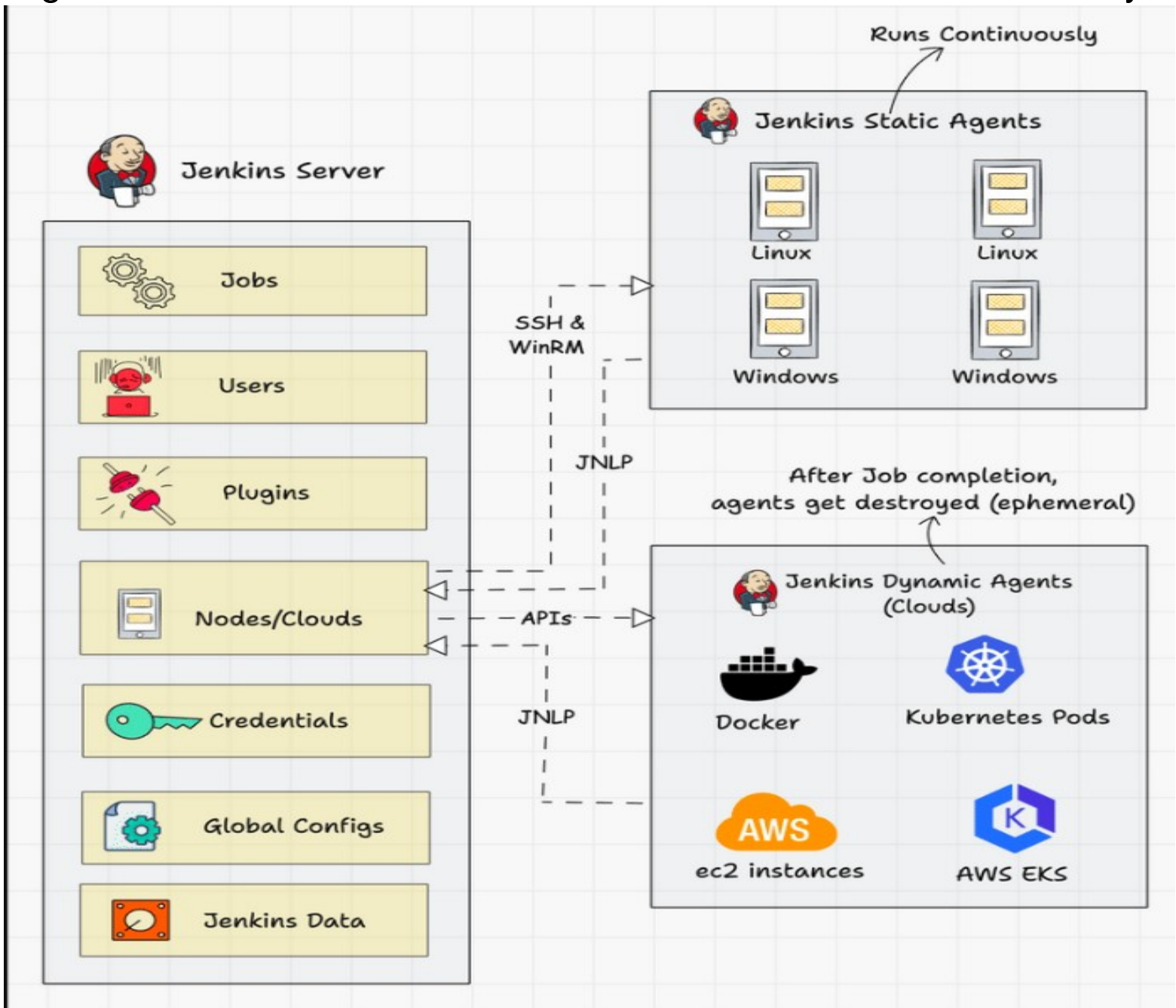
- Jenkins' extensive plugin ecosystem allows it to integrate with virtually any tool used in software development, from version control systems like Git to issue tracking systems like JIRA, and artifact repositories like Artifactory.

8. Security and Compliance

- Jenkins can help enforce security policies and compliance standards by automating security scans and compliance checks.

- Static Code Analysis: Plugins can scan source code for security vulnerabilities.
- Compliance Checks: Jenkins can run scripts or tools that check compliance with various standards.

The following diagram shows the overall architecture of Jenkins and the connectivity workflow..

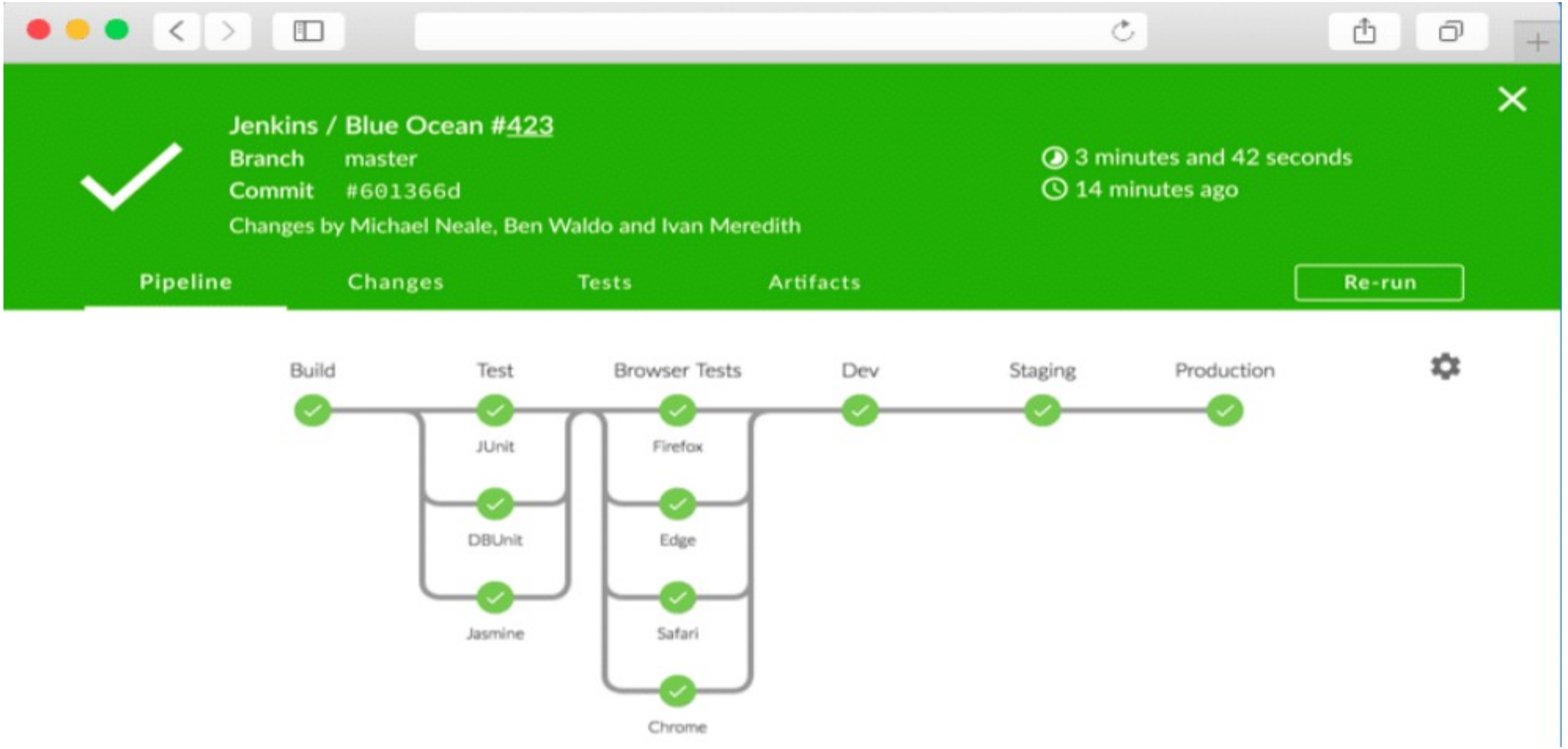


Following are the key components in Jenkins

- Jenkins Master Node:
  - Jenkins’s server or master node holds all key configurations.
  - Jenkins master server is like a control server that orchestrates all the workflow defined in the pipelines.
  - For example, scheduling a job, monitoring the jobs, etc.
- Jenkins Agent Nodes/Clouds:
  - Jenkins agents are the worker nodes that actually execute all the steps mentioned in a Job.
  - When you create a Jenkins job, you have to assign an agent to it.
  - Every agent has a label as a unique identifier.
- Jenkins Web Interface

Jenkins Web Interface :

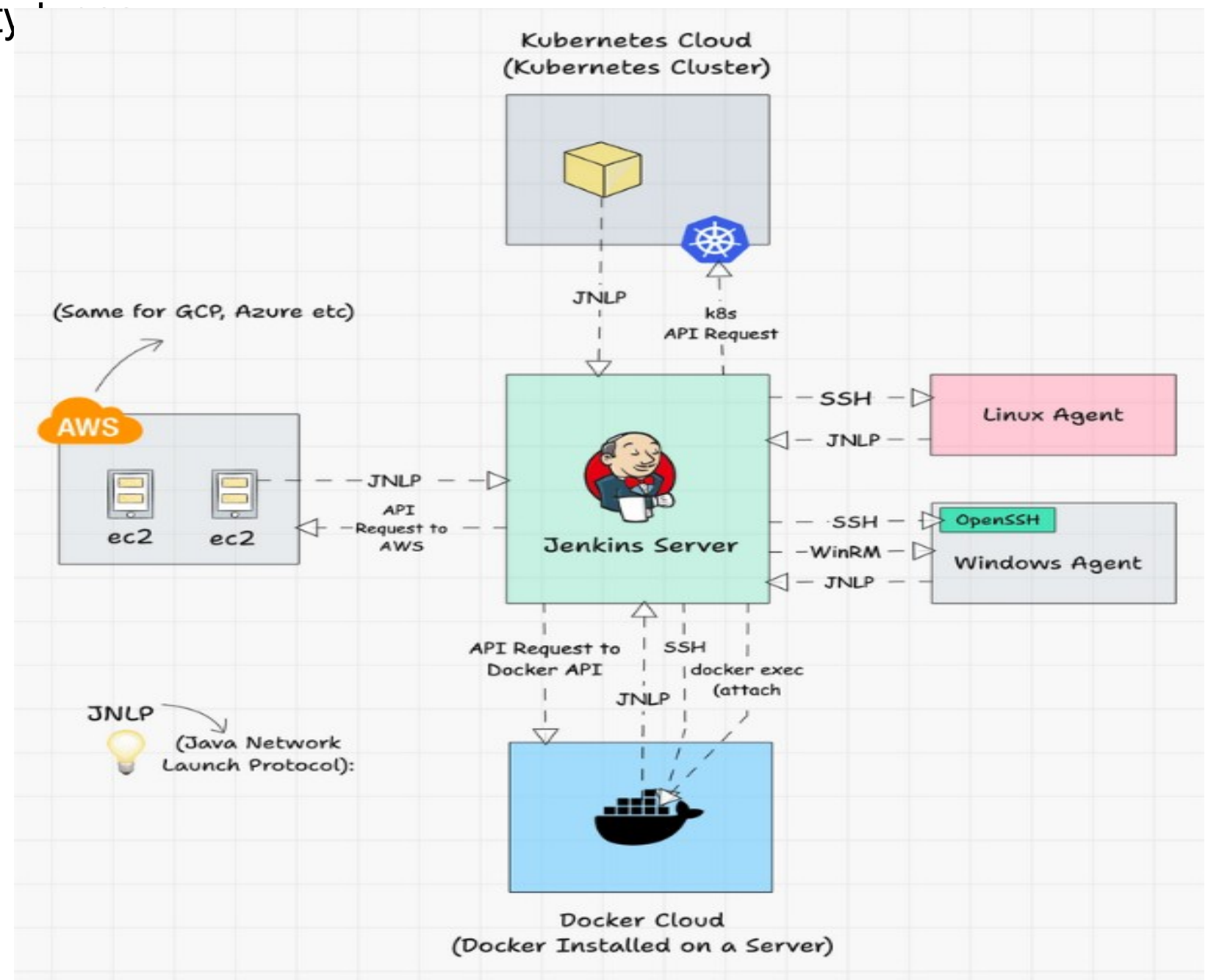
Jenkins 2.0 introduced a very intuitive web interface called “Jenkins Blue Ocean“. It has a good visual representation of all the pipelines.



Jenkins server-agent Connectivity

- There are two types of Jenkins agents
  - **Agent Nodes**: These are servers (Windows/Linux) that will be configured as static agents.These agents will be up and running all the time and stay connected to the Jenkins server. Organizations use custom scripts to shut down and restart the agents when is not used. Typically during nights & weekends.
  - **Agent Clouds**: Jenkins Cloud agent is a concept of having dynamic agents. Means, whenever you trigger a job, a agent gets deployed as a VM/container on demand and gets deleted once the job is completed. This method saves money in terms of infra cost when you have a huge Jenkins ecosystem and continuous builds.
- You can connect a Jenkins master and agent in two ways :
  - **Using the SSH method**:
    - Uses the ssh protocol to connect to the agent. The connection gets initiated from the Jenkins master.
    - There should be connectivity over port 22 between master and agent.
  - **Using the JNLP method**:
    - Uses java JNLP protocol (Java Network Launch Protocol). In this method, a java agent gets initiated from the agent with Jenkins master details. For this, the master nodes firewall should allow connectivity on specified JNLP port.
    - Typically the port assigned will be 50000. This value is configurable.

The following image shows a high-level view of different types of agents and connectivity



Jenkins link

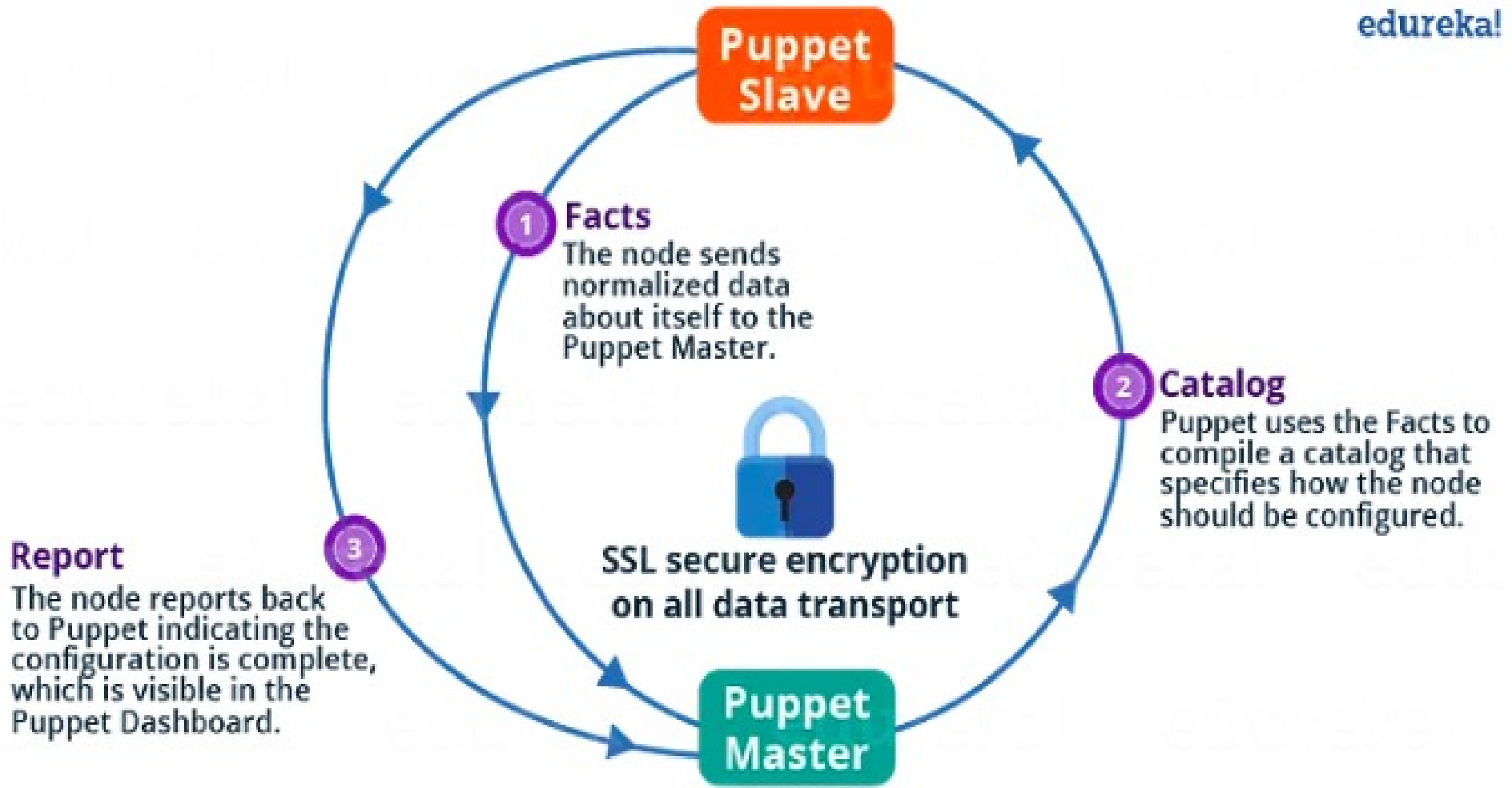
https://www.jenkins.io/

Puppet

- Puppet has established itself as one of the leading services in DevOps for multiple server management.
- Traditionally System administrators utilized shell scripting for operating the servers.
- Still, it was difficult to scale as this requires constant updates to shell scripts for thousands of changing servers and their system configurations.
- Specific DevOps applications like Puppet and Chef have solved this problem by allowing automated server setup, program installation, and system management.

- Puppet is an efficient system management tool for centralizing and automating the configuration management process.
- It can be used as a software deployment tool .
- It can also be utilized as open-source configuration management for server configuration, management, deployment, and orchestration.
- Puppet is specially designed to manage the configuration of Linux and Windows systems.
- It is written in Ruby and uses its unique Domain Specific Language (DSL) to describe system configuration.
- **Configuration management** is maintaining software and computer systems (servers, storage, networks) in a known, desired, and consistent state. It also allows access to an accurate historical record of the system state for project management and audit purposes.
- There are two main versions of Puppet available in the market:
  - **Open Source Puppet:** This is a basic version of the Puppet configuration management tool. It is licensed under the Apache 2.0 system and can be downloaded from Puppet's website.
  - **Puppet Enterprise:** is a paid version that allows enterprises to manage nodes effectively with features like compliance reporting, orchestration, role-based access control, GUI, API, and command-line tools.

Architecture



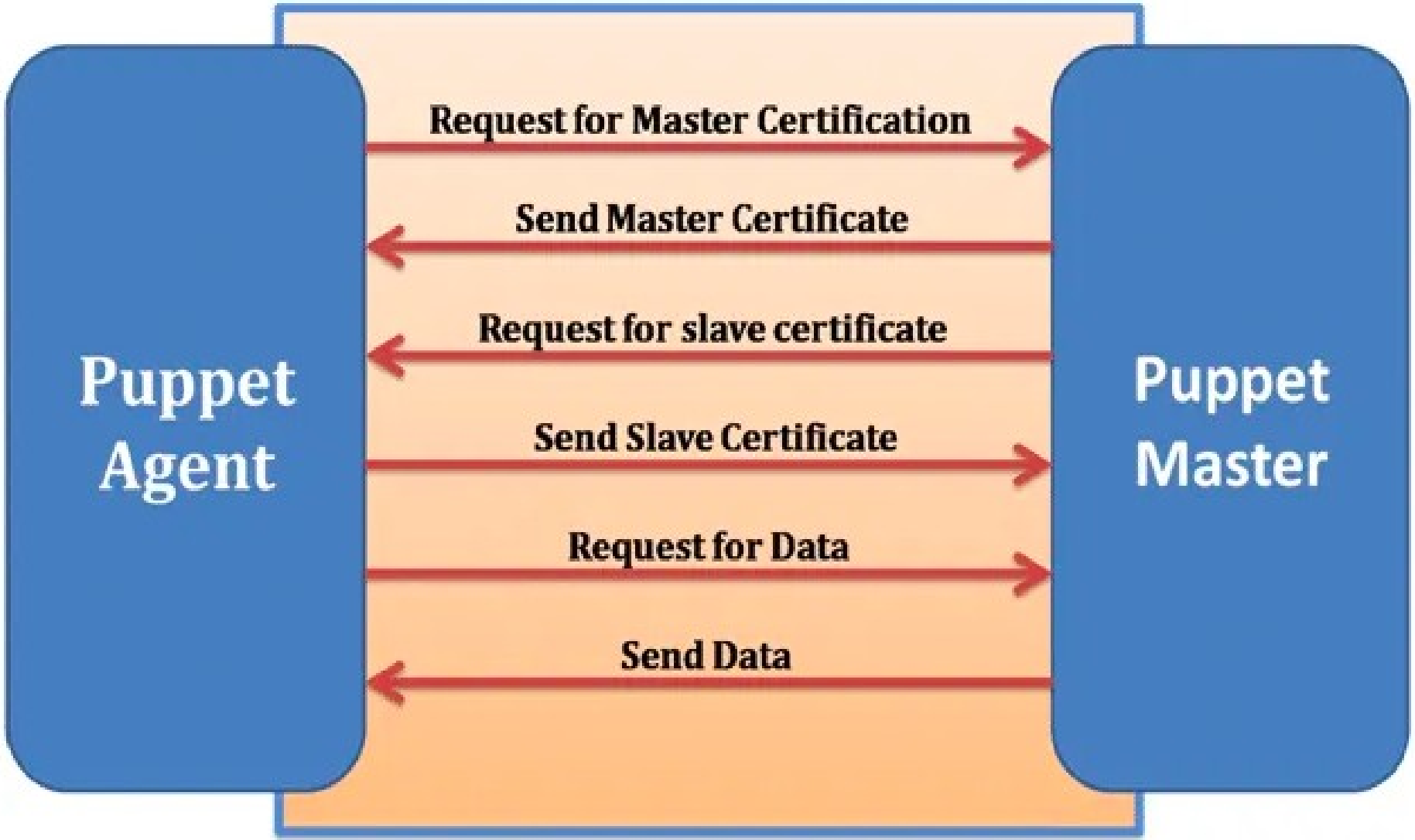
- **Server-agent communication follows this pattern:**

- Puppet follows pull-based architecture where there is an agent who polls the master server and if found any changes in that then it will notify the host machine and the host machine then pull those changes.
- Puppet uses a Master-Slave architecture in which the Master and Slave communicate through a secure encrypted channel with the help of SSL.
- The Puppet Agent sends the Facts to the Puppet Master. Facts are basically key/value data pair that represents some aspect of a Slave state, such as its IP address, uptime, operating system, or whether it’s a virtual machine.
- Puppet Master uses the facts to compile a Catalog that defines how the Slave should be configured. The catalog is a document that describes the desired state for each resource that the Puppet Master manages on a Slave.
- Puppet Slave reports back to Master indicating that Configuration is complete, which is visible in the Puppet dashboard.

Master-Slave communication:

- Puppet is configured in an agent-server architecture, in which a primary server node controls configuration information for a fleet of managed agent nodes.

- **Communication and security in agent-server installations:**
- Puppet Master and Slave communicate through a secure encrypted channel with the help of SSL.
- primary servers and agents communicate by HTTPS using SSL certificates.
- includes a built-in certificate authority for managing certificates. Agents automatically request certificates through the primary server's HTTP endpoint, and you use the puppetserver ca command to inspect requests and sign new certificates.


Puppet Agent
Request for Master Certification
Send Master Certificate
Request for slave certificate
Send Slave Certificate
Request for Data
Send Data
Puppet Master

Components of puppet:

- **Manifest File:**
  - Every Slave has got its configuration details in Puppet Master, written in the native Puppet language.
  - These details are written in the language which Puppet can understand and are termed Manifests.
  - They are composed of Puppet code and their filenames use the .pp extension.
  - These are basically Puppet programs.

- **Module:**
  - A Puppet Module is a collection of Manifests and data (such as facts, files, and templates), and they have a specific directory structure.
  - Modules are useful for organizing your Puppet code because they allow you to split your code into multiple Manifests.
  - Modules are self-contained bundles of code and data.

Components of puppet:

- **Resource:**
  - Resources are the fundamental unit for modeling system configurations.
  - Each Resource describes some aspect of a system, like a specific service or package.
- **Facter:**
  - Facter gathers basic information (facts) about Puppet Slaves such as hardware details, network settings, OS type and version, IP addresses, MAC addresses, SSH keys, and more. These facts are then made available in Puppet Master's Manifests as variables.
- **Catalogs:**
  - A Catalog describes the desired state of each managed resource on a Slave. It is a compilation of all the resources that the Puppet Master applies to a given Slave, as well as the relationships between those resources.
- **Mcollective:**
  - It is a framework that allows several jobs to be executed in parallel on multiple Slaves.

Why do we use Puppet?

- **Large installed base:** Puppet is used by more than 30,000 companies worldwide
- **Large developer base:** Puppet is so widely used that lots of people develop it. Puppet has many contributors to its core source code.
- **Long commercial track record:** Puppet has been in commercial use since 2005, and has been continually refined and improved.
- **Platform support:** Puppet Server can run on any platform that supports Ruby. for eg: CentOS, Microsoft Windows Server, Oracle Enterprise Linux, etc.

Applications of Puppet:

- **Speed of Recovery** — The production operations team can rapidly deploy the right configuration to the right box. If a system gets inappropriately reconfigured Puppet will automatically revert it back to a last stable state, or provide the details necessary to manually remediate a system rapidly.
- **The speed of Deployment** — Puppet has provided significant time savings in the way the operations team delivers services for the gaming studios.
- **Consistency of Servers** — Puppet's model-driven framework ensures consistent deployments.
- **Collaboration** — Having a model-driven approach makes it easy to share configurations across the organization, enabling developers and operations teams to work together to ensure new service delivery is of extremely high quality.

Puppet Link

https://www.puppet.com/

- Ansible is a software tool that provides simple but powerful automation for cross-platform computer support.
- It is primarily intended for IT professionals, who use it for application deployment, updates on workstations and servers, cloud provisioning, configuration management, intra-service orchestration, and nearly anything a systems administrator does on a weekly or daily basis.
- Ansible doesn't depend on agent software and has no additional security infrastructure, so it's easy to deploy.
- Because Ansible is all about automation, it requires instructions to accomplish each job.
- With everything written down in simple script form, it's easy to do version control.
- Ansible allows you to configure not just one computer, but potentially a whole network of computers at once, and using it requires no programming skills.
- Instructions written for Ansible are human-readable.
- Whether you're entirely new to computers or an expert, Ansible files are easy to understand.

Ansible

- Ansible is a powerful and widely adopted automation and configuration management tool important in 2024 for several reasons.
- This tool stands out for its simplicity and versatility.
- It empowers organizations to automate repetitive tasks, provisioning of infrastructure, and configuration management across diverse environments, making it an invaluable asset for DevOps and IT teams.
- Furthermore, Ansible's agentless architecture, declarative language, and a vast library of pre-built modules make it accessible to both beginners and seasoned professionals.
- As organizations prioritize efficiency, scalability, and the rapid deployment of applications and services,
- Ansible remains an indispensable DevOps toolkit, helping teams streamline operations, enhance security, and maintain infrastructure at scale, all while reducing manual errors and increasing agility in a fast-paced technological landscape.

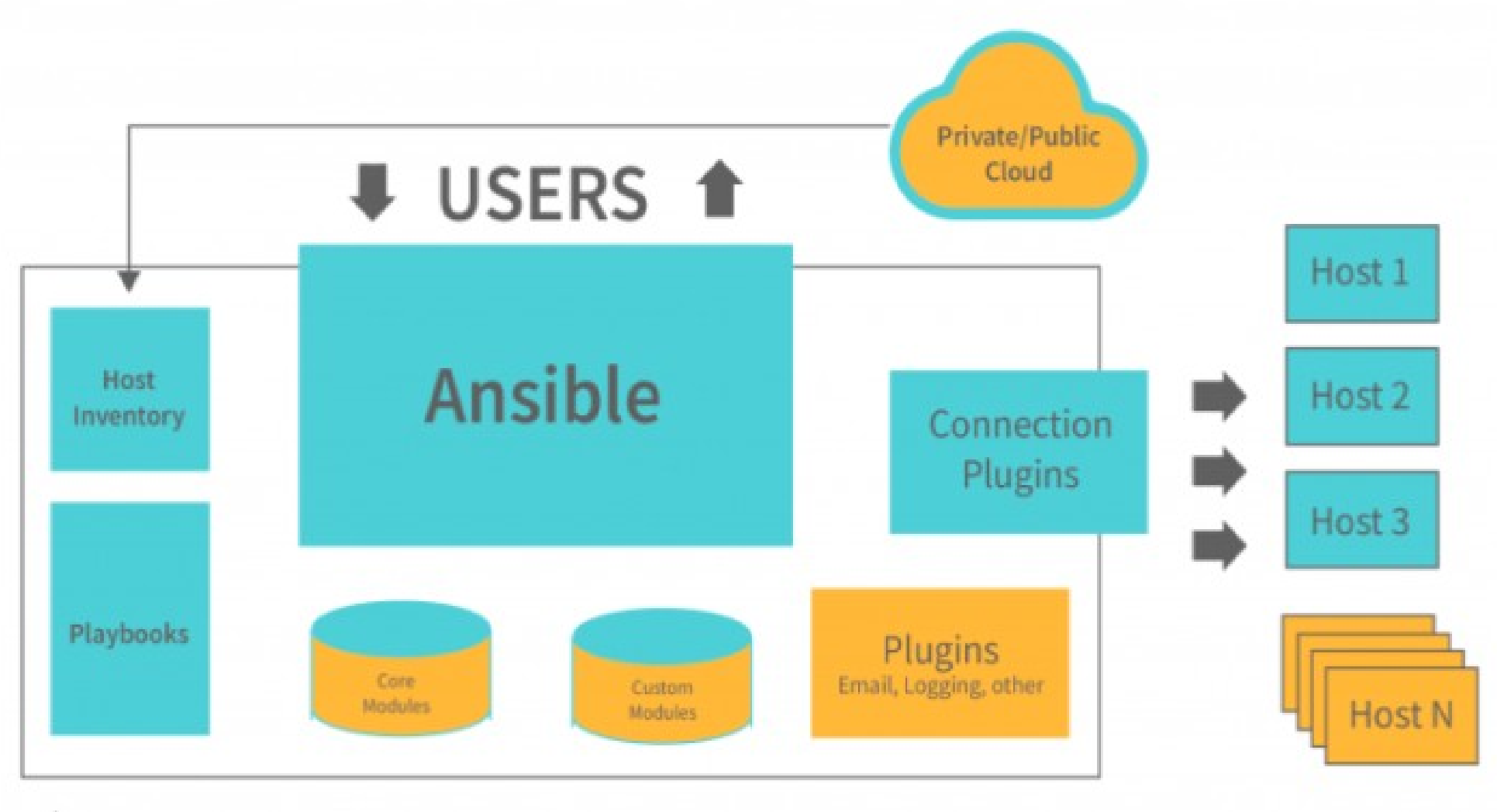
Understanding Ansible’s Architecture

- At its core, Ansible is built on a simple yet powerful architecture, primarily composed of two types of machines: the Control Node and Managed Nodes.
- Control Node: The machine where Ansible is installed and from which all tasks and playbooks are run. You can have multiple control nodes.
- Managed Nodes: The servers (nodes) you manage with Ansible. There’s no need to install Ansible on these nodes, as the control node communicates with them using SSH or PowerShell.

Components of Ansible

- **Modules**:
  - Modules are script-like programs written to specify the desired state of the system. These are typically written in a code editor.
  - Modules are written by the developer and executed via SSH.
  - Modules are part of a larger program called Playbook.
  - Ansible module is a standalone script that can be used inside an Ansible Playbook.
- **Plugins**:
  - Plugins are pieces of code that enhance the core functionality of Ansible. Plugins execute on the control node.
- **Inventory:**
  - Ansible reads information about the machines you manage from the inventory. Inventory is listed in the file which contains IP addresses, databases, and servers.
- **Playbook:**
  - Playbooks are files written in YAML. Playbooks describe the tasks to be done by declaring configurations in order to bring a managed node into the desired state.

Architecture



Architecture

- Everything from servers to desktop computers to network configurations and applications can all be stored in a Public/Private Cloud.
- It demonstrates that it has all the features of a sizable cloud platform, but it also allows users to communicate with all of the modules and API. It also demonstrates that it has security measures.
- An inventory file contains devices, groups, host variables, templates, and tasks. Along with the inventory file, playbooks must be created that describe how to handle all the devices, groups, host variables, and templates.
- Once Ansible has a task or set of tasks in a playbook to run, it needs to know where to run those tasks. It needs an inventory of hosts. Ansible has a concept called “inventories” that consist of lists of hosts to perform actions stored in files as YAML.
- Modules in Ansible allow you to interact with the cloud like Azure or on-prem resources like Hyper-V. An Ansible module is a standalone, reusable script that can be used with the Ansible API. They can also be used by Ansible playbooks. One of the key factors of an Ansible module is that they are reusable and not meant for just one environment.
- The augmented ansible’s core is created by combining all of the cache, logging purposes, ansible’s functioning, and so on.
- It allows for the creation of various agentless frames that can be utilized in multiple automated networks.
- A single repository can contain all the computers of an operational or IT infrastructure network.
- Ansible is being used to make Linux and Unix machines automated.
- It provides the needed APIs for the interaction of the end-to-end modules

Link for ansible

https://www.ansible.com/

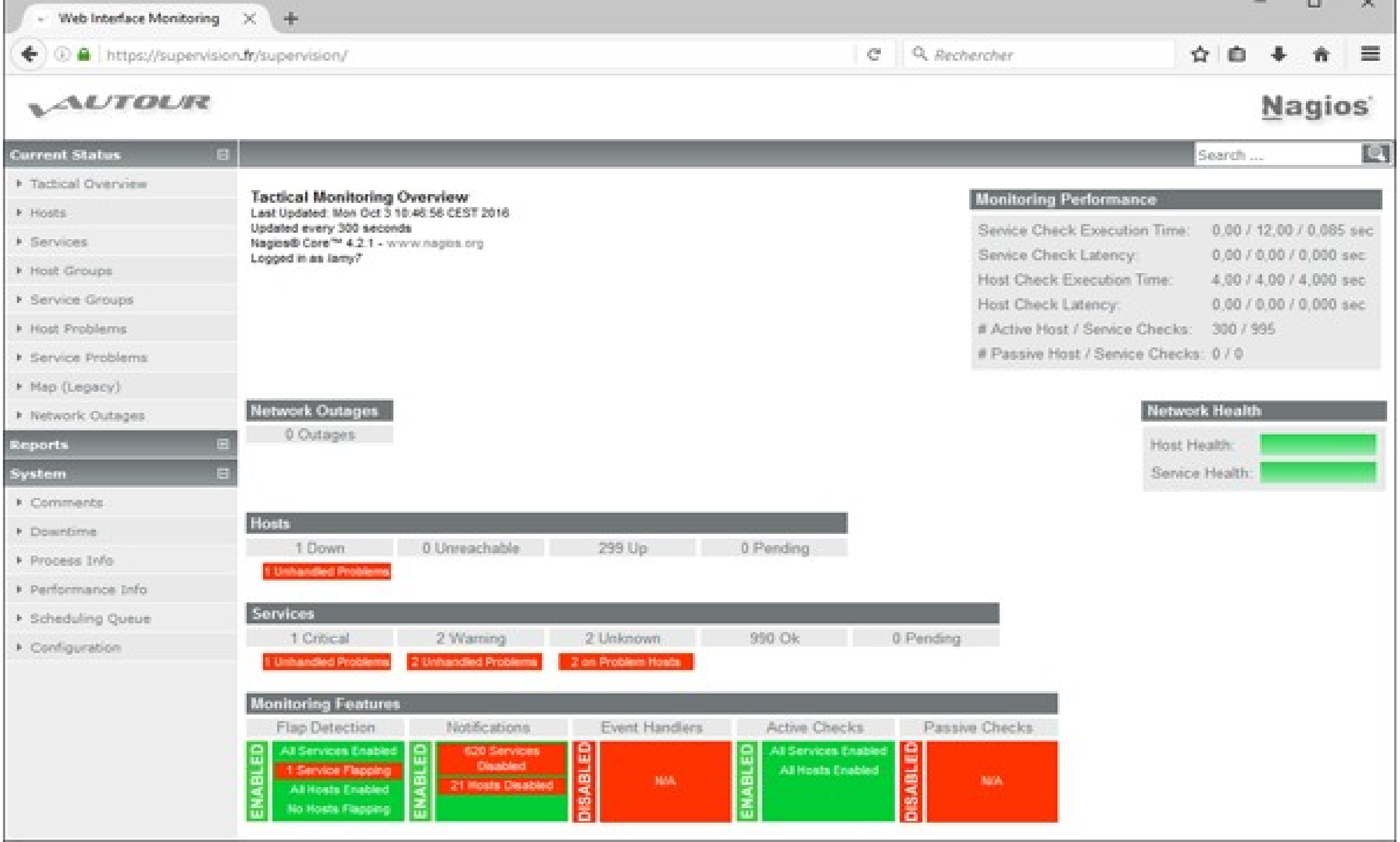
Continuous monitoring

- Continuous monitoring is a mechanism for detecting, reporting, and responding to all attacks on the infrastructure.
- The task of constant monitoring is started once the application is deployed to the server.
- The entire process revolves around maintaining and responding to the company’s infrastructure.
- When the production servers are deployed, continuous monitoring begins.

Need of Nagios

Nagios is needed to **monitor, manage, and protect IT infrastructure**. It helps organizations keep their systems running smoothly and avoid unexpected failures.

- **Key Needs / Importance of Nagios**
- **Early Problem Detection**
  - Identifies issues in servers, networks, and applications **before they become serious**.
- **24/7 System Monitoring**
  - Continuously monitors **servers, switches, routers, databases, and services**.
- **Reduced Downtime**
  - Sends **instant alerts** (email/SMS) when something goes wrong, helping quick fixes.
- **Performance Monitoring**
  - Tracks CPU usage, memory, disk space, bandwidth, and service response time.
- **Centralized Monitoring**
  - One dashboard to monitor the **entire IT infrastructure**.
- **Improved Reliability**
  - Ensures critical services like web servers, mail servers, and databases are always available.
- **Cost Effective**
  - Open-source and customizable, reducing dependency on expensive tools.
- **Scalability**
  - Suitable for **small networks to large enterprise systems**.
- **Security Support**
  - Helps detect abnormal behavior that may indicate **security threats**.
- **Better IT Decision Making**
- Provides logs and reports for **capacity planning and system improvement**.


Web Interface Monitoring
https://supervision.fr/supervision/
Rechercher
AUTOUR
Nagios
Current Status
Tactical Overview
Hosts
Services
Host Groups
Service Groups
Host Problems
Service Problems
Map (Legacy)
Network Outages
Reports
System
Comments
Downtime
Process Info
Performance Info
Scheduling Queue
Configuration
Search ...
Tactical Monitoring Overview
Last Updated: Mon Oct 3 18:46:56 CEST 2016
Updated every 300 seconds
Nagios® Core™ 4.2.1 - www.nagios.org
Logged in as lamy7
Monitoring Performance
Service Check Execution Time: 0.00 / 12.00 / 0.085 sec
Service Check Latency: 0.00 / 0.00 / 0.000 sec
Host Check Execution Time: 4.00 / 4.00 / 4.000 sec
Host Check Latency: 0.00 / 0.00 / 0.000 sec
Active Host / Service Checks: 300 / 995
Passive Host / Service Checks: 0 / 0
Network Outages
0 Outages
Network Health
Host Health:
Service Health:
Hosts
1 Down
0 Unreachable
299 Up
0 Pending
1 Unhandled Problems
Services
1 Critical
2 Warning
2 Unknown
990 Ok
0 Pending
1 Unhandled Problems
2 Unhandled Problems
2 on Problem Hosts
Monitoring Features
Flap Detection
Notifications
Event Handlers
Active Checks
Passive Checks
ENABLED
All Services Enabled
1 Service Flapping
All Hosts Enabled
No Hosts Flapping
ENABLED
820 Services Disabled
21 Hosts Disabled
DISABLED
N/A
ENABLED
All Services Enabled
All Hosts Enabled
DISABLED
N/A

History

- 1996-Ethan Galstad builds a new application for Linux OS based on the ideas and design of his previous work.
- In 1999 the plugins that were initially released as part of the NetSuite distribution are now available as a separate project called Nagios Plugins.
- Due to copyright problems with the name “NetSaint,” Ethan renamed the project “Nagios” in 2002.
- In June 2005, Nagios was named Project of the Month on SourceForge.net.
- Nagios Enterprises launches Nagios XI, the first commercial version, in 2009.
- In 2016 the foundation of Nagios had been downloaded over 7,500,000 times directly from the SourceForge.net website.

Features

- Nagios is a customizable, functional, and safe monitoring system.
- It has a good log system.
- Nagios has a user-friendly and informative web interface, and it sends out updates when a condition changes.
- Using Nagios, you can track the entire business process and IT infrastructure in a single pass.
- Writing new plugins in the language of your choice is simple, thanks to the product’s architecture.

Nagios

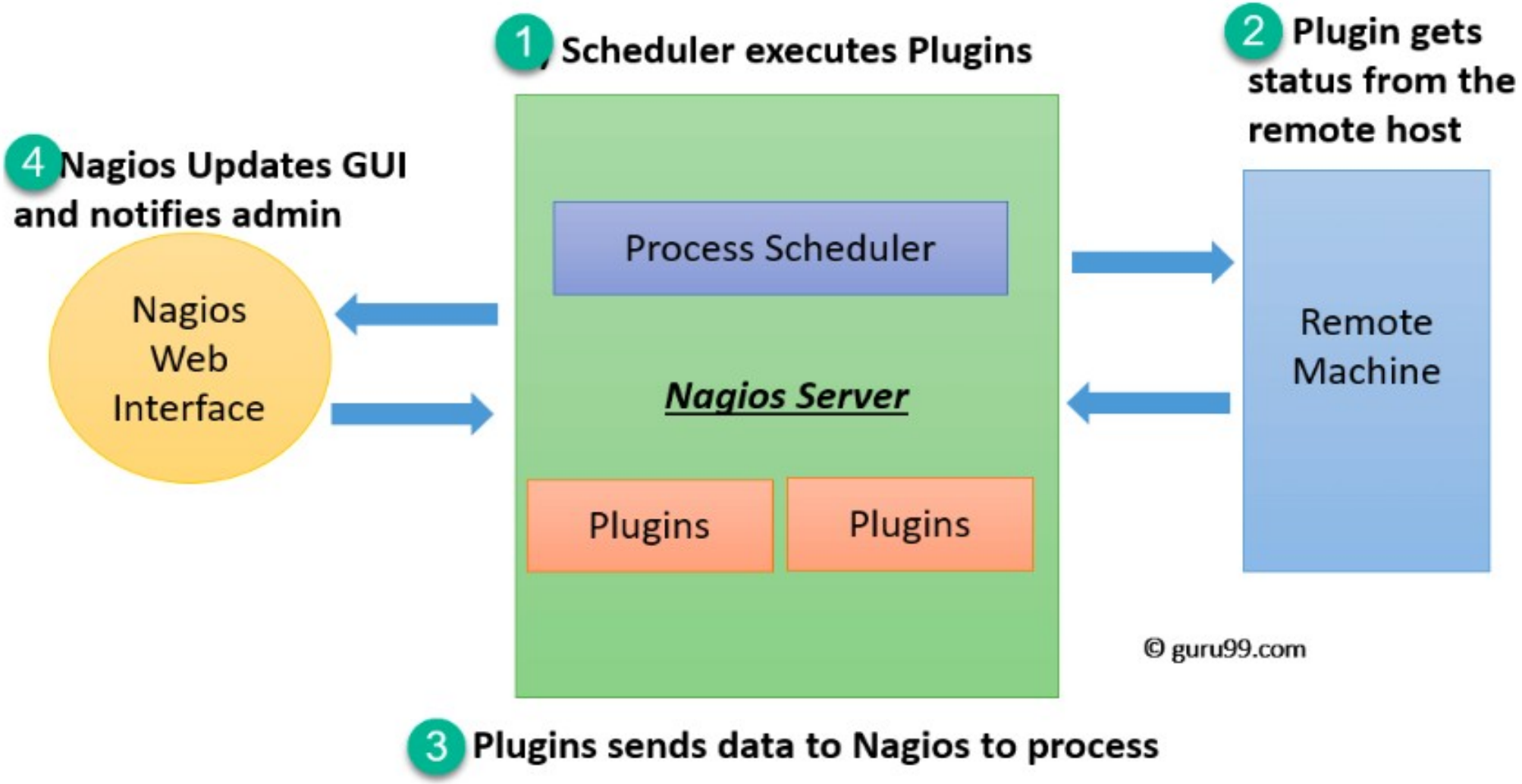
- Nagios, an open-source monitoring and alerting system, remains vital due to its enduring significance in maintaining the reliability and performance of IT infrastructure and applications.
- Organizations increasingly rely on complex systems and services.
- Nagios plays a crucial role by providing real-time monitoring and alerting capabilities, allowing IT teams to detect and address issues before they impact users or cause system outages.
- Its versatility, extensibility, and support for both on-premises and cloud environments make Nagios a valuable tool for ensuring critical systems' availability, stability, and security, aligning perfectly with the demands of modern IT operations and DevOps practices.

- It was designed to run on the Linux operating system and can monitor devices running Linux, Windows and.
- Nagios software runs periodic checks on critical parameters of application, network and server resources.
- For example, Nagios can monitor memory use, disk use and microprocessor load, as well as the number of currently running processes and log files.
- Nagios also can monitor services such as Simple Mail Transfer Protocol (SMTP), Post Office Protocol 3, Hypertext Transfer Protocol (HTTP) and other common network protocols.

Architecture

- **Scheduler runs checks**
- The **Nagios server** has a scheduler.
- It decides **when to check** servers and services and runs small programs called **plugins**.
- **Plugin checks the remote machine**
- The plugin goes to the **remote machine** (server, router, service).
- It checks things like **CPU, disk, memory, or service status**.
- **Plugin sends result back to Nagios**
- After checking, the plugin sends the result (**OK** / **Warning** / **Critical**) back to the **Nagios server**.
- **Nagios updates screen and alerts admin**
- Nagios shows the status on the **web interface (GUI)**.
- If there is a problem, it **notifies the administrator** (email/SMS).

Architecture



Nagios Architecture

- The scheduler is a component of server part of Nagios.
- It sends a signal to execute the plugins at the remote host.
- The plugin gets the status from the remote host
- The plugin sends the data to the process scheduler
- The process scheduler updates the GUI and notifications are sent to admins
- Plugins:
  - Nagios plugins provide low-level intelligence on how to monitor anything and everything with Nagios Core.
  - Plugins operate acts as a standalone application, but they are designed to be executed by Nagios Core.
  - It connects to Apache that is controlled by CGI to display the result.
  - Moreover, a database connected to Nagios to keep a log file.

Application of Nagios

- The Nagios application monitoring tool is a health check and monitoring device for a typical Data Center, and it includes a wide range of equipment, including Nodes in the server and network; from a single console, you can control all of your applications.
- With transaction-level insights, you can track your application, Middleware and Messaging Components can Be Monitored, Reports and dashboards that can be customized, Backup Power Supply (UPS).

Link for Nagios

https://www.nagios.org/

Kubernetes

- **Kubernetes** is an **open-source platform** used to **automatically manage, run, scale, and monitor containerized applications**.

Kubernetes is a container orchestration system that manages deployment, scaling, and operation of containerized applications.

Orchestration means **automatically arranging, managing, and controlling many tasks or systems so they work together smoothly**.

- If one application container **crashes**, Kubernetes **restarts it automatically**.
- If many users access an app, Kubernetes **adds more containers** to handle the load.

- Kubernetes is an open-source platform that automates the management, scaling, and deployment of containerized applications:
- Features:
- Kubernetes deploy applications
- Roll out changes to applications
- Scale applications up and down
- Monitor applications
- Automatically manage service discovery
- Incorporate load balancing
- Track resource allocation

Benefits of Kubernetes

Automated operations

Kubernetes has built-in commands to handle a lot of the heavy lifting that goes into application management, allowing you to automate day-to-day operations. You can make sure applications are always running the way you intended them to run.

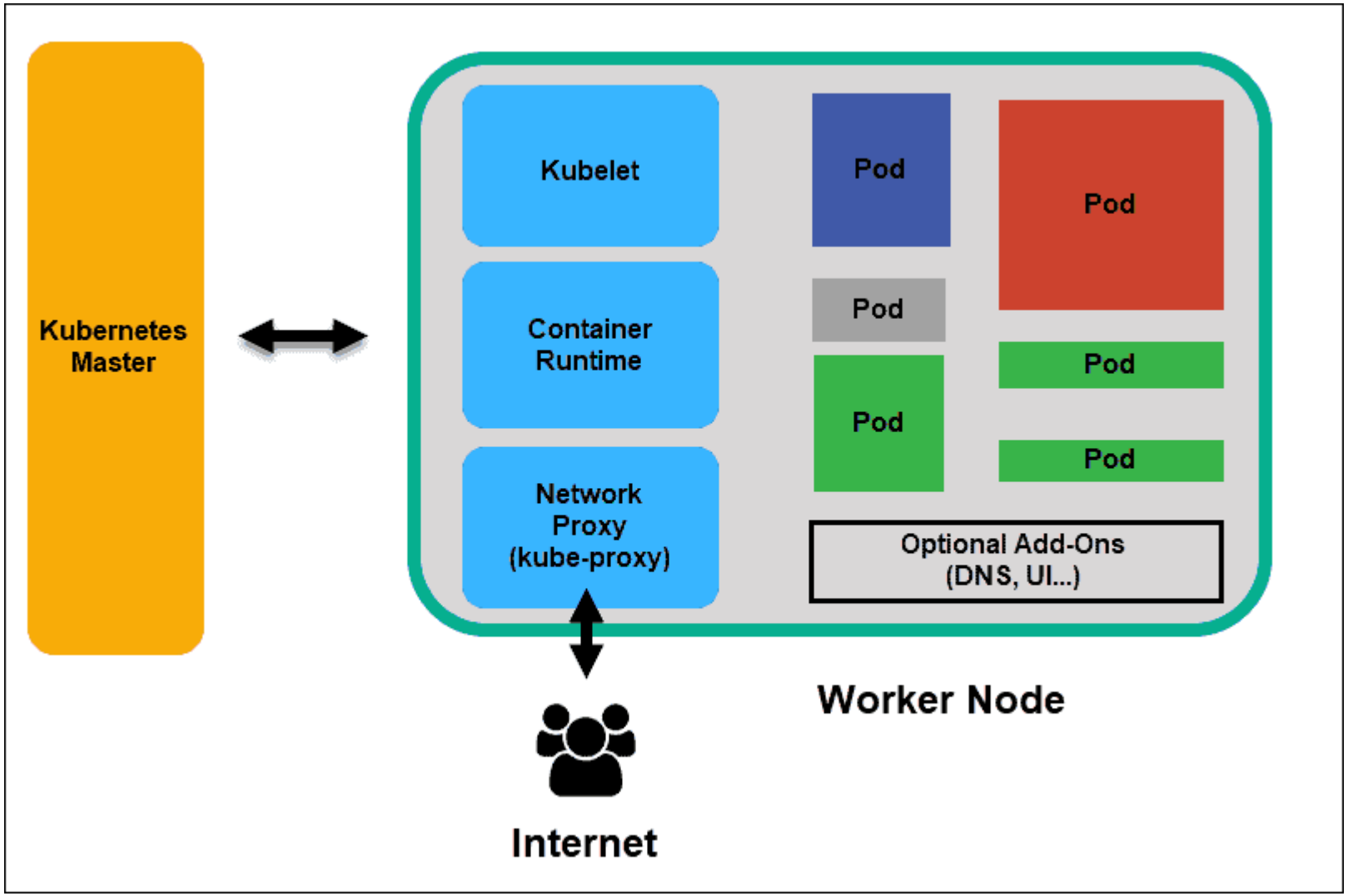
Infrastructure abstraction

When you install Kubernetes, it handles the compute, networking, and storage on behalf of your workloads. This allows developers to focus on applications and not worry about the underlying environment.

Service health monitoring

Kubernetes continuously runs health checks against your services, restarting containers that fail, or have stalled, and only making available services to users when it has confirmed they are running.

Architecture



Node Components

- Run on every node, maintaining running pods and providing the Kubernetes runtime environment:
- **kubelet** : Ensures that Pods are running, including their containers.
- **kube-proxy (optional)**: Maintains network rules on nodes to implement Services.
- **Container runtime :** Software responsible for running containers. Read Container Runtimes to learn more.

- Your cluster may require additional software on each node; for example, you might also run systemd on a Linux node to supervise local components.

Link

https://kubernetes.io/docs/concepts/architecture/

Maven

- Due to its enduring significance in managing project dependencies, building, and project lifecycle management,
- Maven remains a pivotal tool in SD and DevOps.
- As a robust build automation and project management tool, Maven simplifies the complexities of Java-based project development by streamlining the compilation, testing, packaging, and distribution processes.
- It ensures consistent and reproducible builds, making it easier for development teams to collaborate efficiently and deliver high-quality software.
- Maven's role in managing dependencies and facilitating continuous integration and deployment remains crucial. Its ability to handle complex build scenarios and integrate seamlessly with modern DevOps practices makes it indispensable for ensuring software projects' reliability, maintainability, and scalability in 2024 and beyond.